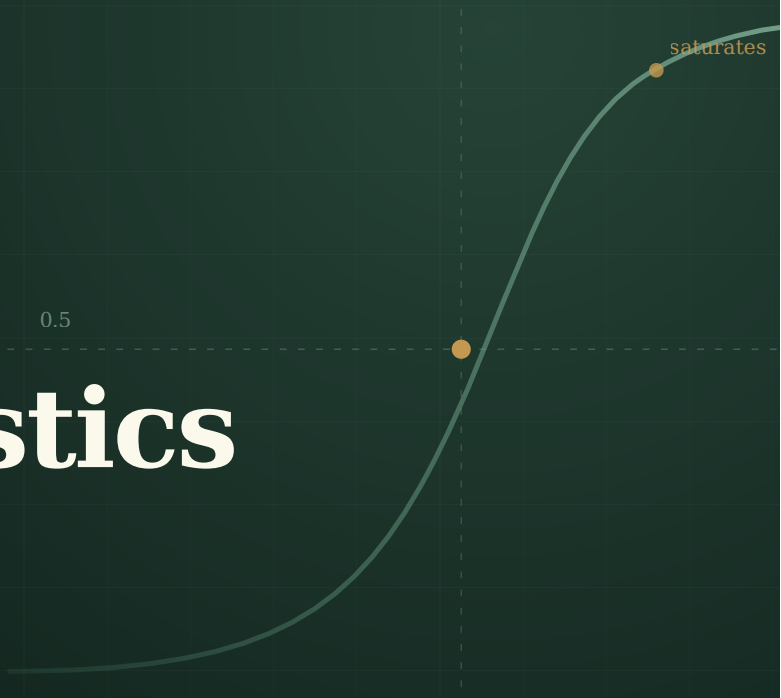


A WORKING FIELD GUIDE

From Statistics to *Neural Processes*

The minimum statistics you need, the IRT model underneath the competition, and how Neural Processes learn to predict with a handful of labels — each idea with an intuition and a solved micro-example.

30 concepts · **30 solved micro-examples** · **6 parts**

Built from the working repository `agrov/aims-neurips`

Start with the 45-minute path → page 2

Reading time: 45 min · half a day · or one reference page at a time

October 2026
v1.0

AIMS · PAIEC 2026

Contents

START HERE

How to use this guide 2
Math survival kit 3

PART I · THE STATISTICS YOU ACTUALLY NEED

1 One picture of the whole problem 5
2 Probability, odds, and logits 6
3 Likelihood and log loss 7
4 Bayes, priors, and shrinkage 8
5 Uncertainty, paired tests, and honest error bars 9
6 Entropy, KL, and the value of a label 10

PART II · SCORING A PROBABILISTIC PREDICTOR

7 Brier score: the only number you are paid on 11
8 Calibration: are your probabilities honest? 12
9 Discrimination and AUC: can you tell items apart? 12
10 The learning curve and Brier ALC 13
11 Who gets weight? Aggregation and the leaderboard 14

PART III · ITEM RESPONSE THEORY

12 The 2PL model: ability meets difficulty 15
13 Fitting ability: MAP and Newton in five lines 16
14 Amortized IRT: difficulty from content, not a lookup table 17
15 Identifiability, conventions, and leakage 19

PART IV · ADAPTATION WITH ALMOST NO LABELS

16 The Beta-Binomial control that is hard to beat 20
17 Hierarchical shrinkage and empirical Bayes 20
18 Post-hoc calibration in practice 22

PART V · NEURAL PROCESSES

19 A distribution over functions, not a single function 23
20 Sets, pooling, and permutation invariance 23
21 CNP: one deterministic summary 24
22 NP: the latent variable and the ELBO 25
23 ANP: attention makes the summary target-specific 26
24 NP-AIRT in this repo: where the anchor lives 27
25 Training an episodic model without fooling yourself 28
26 Reading the diagnostics: signal, scale, or noise? 28

PART VI · THE COMPETITION PLAYBOOK

27 The model ladder: what each rung actually tests 30
28 Ablations that survive review 31
29 Failure-mode checklist 31
30 Cram sheet 32

APPENDIX · GLOSSARY AND MAP

A Glossary 33
B Where things live in the repo 33
C Reading list 34

Boxes: **Intuition** gives the picture, **Solved** is a 60-second numeric check, **Trap** is the mistake, **In this repo** points at the code.

BEFORE YOU START

How to use this guide

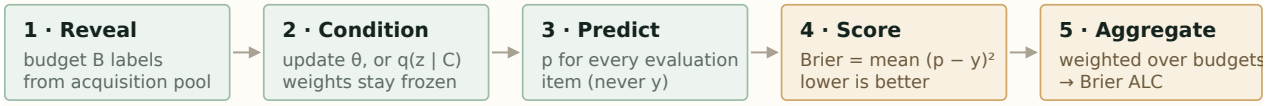
You do not need to read this cover to cover. Pick a path, keep the cram sheet open, and come back when a specific symbol stops making sense.

THREE WAYS IN

PATH	READ	AFTERWARDS YOU CAN
45 minutes the survival path	§1, §7, §10, §12, §19, §21, then §30 (the cram sheet)	Explain the competition at a whiteboard, read a Brier curve, and say what a CNP is without hand-waving.
Half a day the working path	Parts I–III, then §16–18 and §19–24	Follow the paper, read the training code, and debug a model that has good AUC but bad Brier.
Reference the "why is this weird?" path	Any section, Appendix A (the glossary), and the formula sheet	Recover one specific idea in five minutes without re-reading the whole guide.

THE COMPETITION IN ONE PARAGRAPH

A **subject** (an AI system) is evaluated on a **benchmark** of **items** (tasks). For each subject–benchmark pair, some item outcomes are hidden. You may reveal a small number of outcomes from a separate **acquisition pool**; the budgets are $B \in \{0, 1, 3, 7, 15, 31\}$. At every budget you must output a probability p_{sj} that subject s succeeds on evaluation item j. The scorer compares those probabilities with the hidden binary outcomes using the **Brier score**, then averages across budgets with fixed weights. Nothing else is scored. Your job is to turn *content* (item text, metadata, subject metadata) plus a few revealed labels into *honest probabilities* for items you have never seen the outcome of.



Nested prefixes: the 1-label context is contained in the 3-label context, and so on. At B = 0 the model must stand on content alone.

The official flow. Anything that uses the hidden evaluation outcomes, directly or indirectly, is leakage — even if it "only" tunes a calibration constant.

FIVE OBJECTS, ONE SENTENCE EACH

Subject	the AI system being evaluated — a model, an agent, or a model plus tooling.
Item	one task presented to the subject (a coding issue, a math problem, a web task).
Outcome	what actually happened, encoded as $y \in \{0, 1\}$.
Predictor	your submitted program; it estimates outcomes without running the subject.
Prediction	a probability $p \in [0, 1]$; not a label, not a decision, not a ranking.

The subject is not your predictor. Your predictor predicts how the subject will perform.

THREE QUESTIONS EVERY METHOD MUST ANSWER

1. **Item side:** where do difficulty and discrimination come from? A learned table, item text, metadata, or an amortized network $f_\phi(x_j)$?
2. **Subject side:** where does ability come from? A learned table, subject metadata, or the revealed labels?
3. **Update rule:** exactly how do the revealed labels change the prediction? A posterior, a MAP fit, a learned set encoder, or nothing?

If you can answer those three for any model in the repo, you understand it. The rest is bookkeeping.

HOW TO READ THE BOXES

> **INTUITION**
> The plain-language picture. If you only read one thing per section, read this.

SOLVED IN 60 SECONDS

A tiny numeric example you can redo on paper. Do redo it — reading arithmetic is not the same as doing it.

TRAP

A mistake that is easy to make and expensive to debug.

IN THIS REPO

Where the idea appears in the code or docs, and what the current implementation actually does.

NOTATION, ONCE

SYMBOL	MEANS	FIRST USED
C, T	acquired context set; target set	§1
x_i, y_i	item input (no outcome); binary outcome	§1
θ_s	ability of subject s	§12
a_j, b_j	item discrimination; item difficulty	§12
z	global latent variable in a Neural Process	§22
r_i, s_i	deterministic path encoding; latent path encoding	§21–23
ϕ, ψ, ω	learned network parameters	§14
$\sigma(\cdot)$	sigmoid, $\sigma(z) = 1/(1+e^{-z})$	§2
$\mathcal{N}(\mu, \sigma^2)$	Gaussian distribution	§4
$\mathrm{KL}(q \,\|\, p)$	Kullback–Leibler divergence	§6

Logs are natural unless stated. “Nats” are the unit; 1 nat = 1.4427 bits. The competition works in probabilities, so most formulas here end in a probability, not a logit.

REFERENCE · FIVE MINUTES

Math survival kit

Nine things from high-school math that the rest of this guide assumes. Each comes with the smallest possible example.

1 · SUMS, PRODUCTS, AND THE TWO GREEK LETTERS

$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$ adds; $\prod_{i=1}^{n} p_i = p_1 p_2 \cdots p_n$ multiplies. A likelihood over independent observations is a product, which is why we take logs immediately.

SOLVED

For $x = (1, 2, 3)$: $\sum x_i = 6$, $\prod x_i = 6$, and $\sum x_i^2 = 14$. The mean is $6/3 = 2$.

2 · EXPONENTIALS AND LOGARITHMS

$\log$ undoes $e^{(\cdot)}$. The three rules you will use constantly:

$$\log(ab) = \log a + \log b, \qquad \log(a/b) = \log a - \log b, \qquad \log(a^b) = b \log a.$$

Probability lives in $(0, 1)$; log-probability lives in $(-\infty, 0]$. That unbounded scale is why we model *logits* instead of probabilities.

SOLVED

$\log(0.8 \cdot 0.3) = \log 0.8 + \log 0.3 = -0.223 - 1.204 = -1.427$. And $e^{-1.427} = 0.24 = 0.8 \cdot 0.3$.

3 · DERIVATIVES AND THE CHAIN RULE

$\frac{d}{dx} x^2 = 2x$, $\frac{d}{dx} \log x = 1/x$, $\frac{d}{dx} e^x = e^x$. The chain rule: if $L = L(f(x))$, then $dL/dx = (dL/df)(df/dx)$. Every “gradient through the network” is only this, repeated.

The sigmoid has a derivative worth memorizing:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big).$$

SOLVED

At $z = 0$, $\sigma = 0.5$, so $\sigma' = 0.25$. At $z = 2$, $\sigma = 0.881$, so $\sigma' = 0.881 \cdot 0.119 = 0.105$. The curve is steepest at zero.

4 · PROBABILITY RULES

For events A, B: $P(A, B) = P(A \mid B)P(B)$. If independent, $P(A, B) = P(A)P(B)$. Bayes flips the conditioning:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

In this guide, A is "the parameters" and B is "the labels we saw". The denominator is a normalizing constant; the interesting part is $P(B \mid A)P(A)$.

5 · EXPECTATION AND VARIANCE

For a discrete random variable X: $\mathbb{E}[X] = \sum_x x\,P(X = x)$, and $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$.

For a Bernoulli outcome with success probability p: $\mathbb{E}[Y] = p$ and $\mathrm{Var}(Y) = p(1-p)$. The variance is largest at $p = 0.5$ and zero at the extremes.

> **SOLVED**
> Six outcomes $(1, 0, 1, 1, 0, 1)$: mean $4/6 = 0.667$. Sample variance $= 0.267$, which equals $p(1-p) \cdot n/(n-1) = 0.222 \cdot 1.2$.

6 · THE GAUSSIAN

$\mathcal{N}(\mu, \sigma^2)$ has density $\frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$. About 68% of mass lies within one standard deviation, 95% within two. A variance of 4 means a standard deviation of 2 — the units matter.

7 · VECTORS, DOT PRODUCTS, MATRICES

A dot product $w^\top x = \sum_i w_i x_i$ is a weighted sum; it is also a similarity score when both vectors are normalized. A weight matrix $W \in \mathbb{R}^{m \times n}$ maps an n-vector to an m-vector: $y = Wx + b$. Neural networks are stacks of these with a nonlinearity between them.

8 · SOFTMAX, ARGMAX, AND "MAX"

Softmax turns arbitrary scores s_i into positive numbers that sum to one: $\mathrm{softmax}(s)_i = e^{s_i} / \sum_k e^{s_k}$. Argmax returns the index of the largest value; $\max$ returns the value.

> **SOLVED**
> Scores $(0, 1, 2)$: softmax $= (0.090,\ 0.245,\ 0.665)$. The argmax is index 3; the max is 2. Attention is a softmax over context items.

9 · READING AN EQUATION WITHOUT PANIC

1. **Name every symbol.** If one is unnamed, the equation is not yet understood.
2. **Check the output type.** Probability? Real number? Vector? Distribution?
3. **Plug in extremes.** Set $p = 0$, $p = 1$, $\theta = 0$, $n = 0$. Does the formula still make sense?
4. **Check monotonicity.** If ability rises, should the probability rise? Does the formula do that?
5. **Find the units.** A logit is dimensionless; a Brier score is squared probability; a KL is in nats.

> **INTUITION**
> Almost every model in this competition is the same sentence in different clothing: *a subject's ability, compared with an item's difficulty, squashed into a probability*. The Neural Process changes only where ability and difficulty come from, and how labels update them.

PART I

The statistics you actually need

Six ideas carry almost all of the weight: a probability is a prediction, a logit makes it linear, a likelihood scores it, a prior stabilizes it, a standard error tells you whether a difference is real, and entropy tells you what a label is worth.

- 1 · One picture of the problem
- 2 · Probability, odds, logits
- 3 · Likelihood and log loss
- 4 · Bayes, priors, shrinkage
- 5 · Uncertainty and paired tests
- 6 · Entropy, KL, information

1 One picture of the whole problem

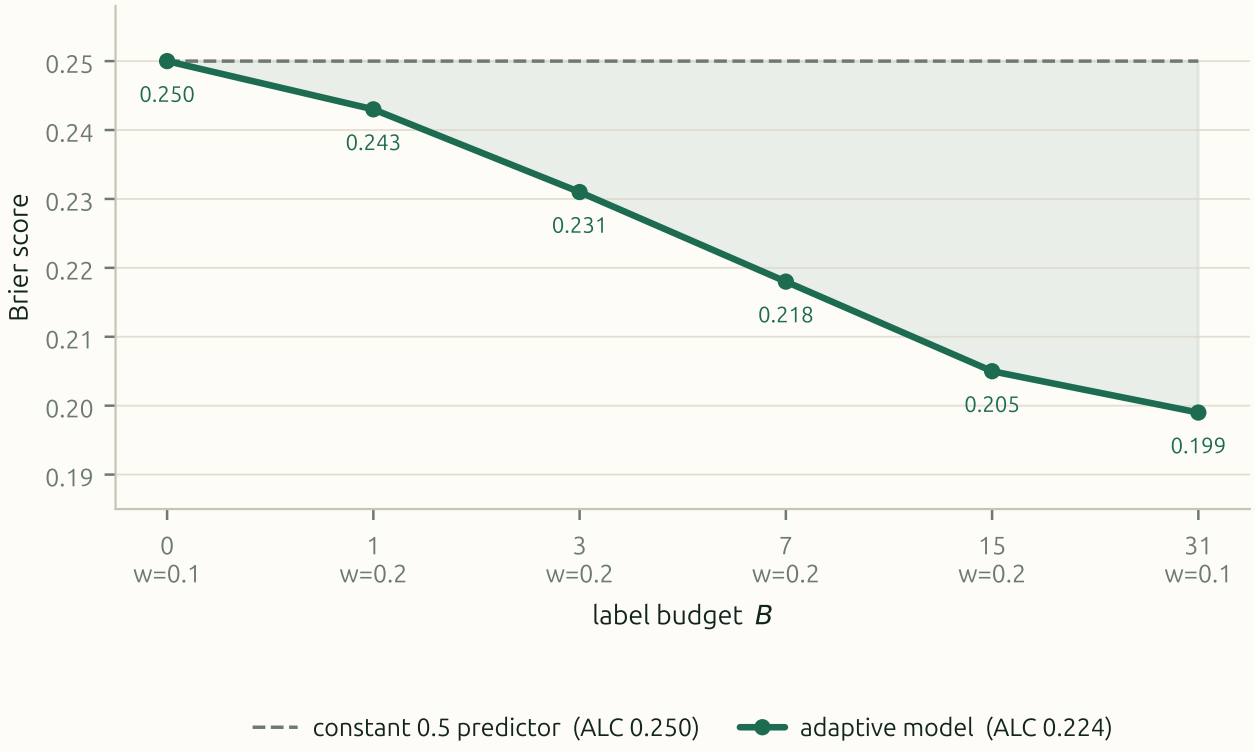
You are filling in a mostly empty subject × item table, and you are paid for the honesty of each filled-in probability, not for the label you would guess.

Imagine a giant table. Each row is a **subject** — one AI system. Each column is an **item** — one task. Each cell would contain a binary outcome $y_{sj} \in \{0, 1\}$: did subject s succeed on item j? In the real data, almost every cell is missing, because running every subject on every item is exactly what evaluation is trying to avoid.

The competition hands you one row-slice at a time: a subject, a benchmark, a set of item descriptions, and two disjoint pools of items. The **acquisition pool** can reveal outcomes, up to the current budget. The **evaluation pool** is where you are scored; its outcomes stay hidden. Your predictor is a function

$$p_{sj} = P\big(y_{sj} = 1 \mid x_j,\ m_s,\ C_s^{(B)}\big),$$

where x_j is everything known about item j before its outcome, m_s is subject metadata, and $C_s^{(B)} = \{(x_i, y_{si})\}_{i=1}^{B}$ is the set of acquired labels. The output must be a probability for *every* evaluation item.



The object you are optimizing. Brier score at each budget, joined into one weighted number. The shaded area is the part of the curve the adaptive model wins; the dashed line is the "no information" predictor $p = 0.5$. Lower is better.

WHY THIS IS HARD IN THREE DIFFERENT WAYS

- **Cold items.** Evaluation items were never seen. Memorizing an item's difficulty is useless; you need difficulty to be a function of item content.

- **Cold subject at $B = 0$.** With no labels, you know only metadata. A subject-level prior has to do all the work.
- **Tiny budgets.** At $B = 1$, one binary outcome is one bit of evidence — and a noisy bit. The model must update without overreacting.

INTUITION

Think of the table as a light-cone problem. The item column tells you how hard the task is; the subject row tells you how capable the system is; the revealed cells are the only place the two meet. A good model is a principled interpolation between "what the content suggests" and "what the few labels prove".

SOLVED IN 60 SECONDS

Four evaluation items have hidden outcomes $y = (1, 0, 1, 0)$.

Predict $p = 0.5$ for all four. Brier $= \frac{1}{4}\left[(0.5)^2 + (0.5)^2 + (0.5)^2 + (0.5)^2\right] = 0.25$.

Now predict $p = (0.9, 0.2, 0.6, 0.4)$. Brier $= \frac{1}{4}(0.01 + 0.04 + 0.16 + 0.16) = 0.0925$.

Read this: 0.25 is not "average", it is the score of a predictor that knows nothing. A model at 0.27 is *worse than admitting ignorance*. This single fact explains most of the debugging in Part VI.

TRAP

"Accuracy" is not the metric. A model can be 80% accurate and badly calibrated; Brier punishes the probability, not the thresholded decision. Never tune a classifier's accuracy and expect the Brier score to follow.

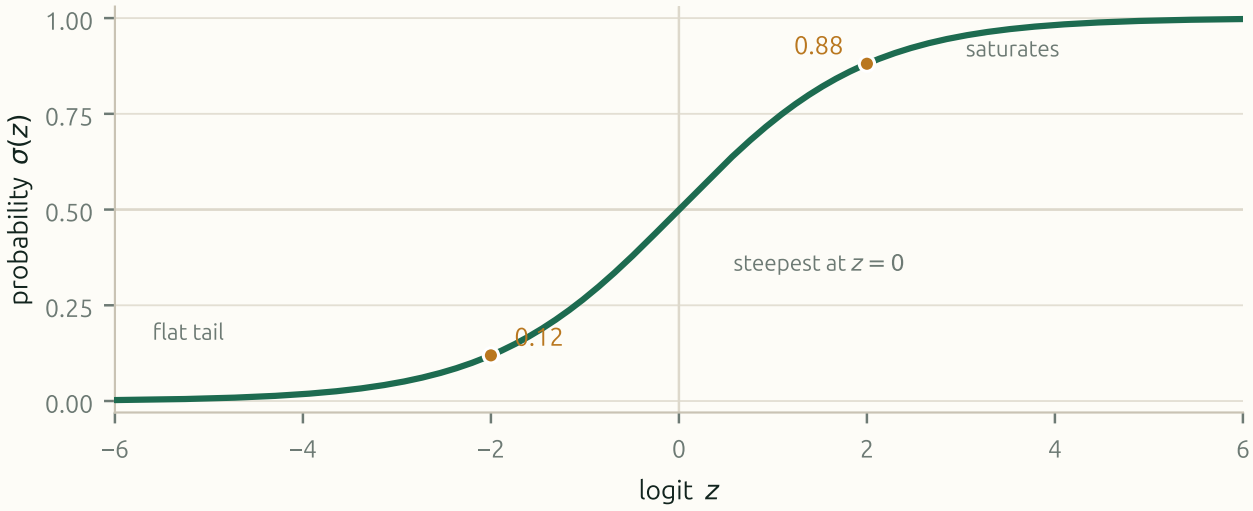
2 Probability, odds, and logits

The logit is the model's native language: it turns "probability" into a number that can be added to, compared, and modeled linearly.

A probability $p \in [0, 1]$ is what you report. The **odds** are $p/(1-p)$ — "four to one" means $p = 0.8$. The **logit** is the log of the odds:

$$z = \operatorname{logit}(p) = \log \frac{p}{1-p}, \qquad p = \sigma(z) = \frac{1}{1+e^{-z}}.$$

The two functions are inverses. $p = 0.5$ maps to $z = 0$; $p \to 1$ maps to $z \to +\infty$; $p \to 0$ maps to $z \to -\infty$. The sigmoid derivative is $\sigma'(z) = \sigma(z)(1-\sigma(z))$, which is why a logit change near zero moves the probability much more than the same change in the tail.



One curve, two scales. The same fitted value is a logit on the horizontal axis and a probability on the vertical axis. Models add evidence in logit space; the scorer sees probability space.

WHY MODEL LOGITS INSTEAD OF PROBABILITIES

- **No boundaries.** A linear model $w^\top x$ can output any real number; the sigmoid keeps the result inside $[0, 1]$.
- **Additive evidence.** Independent pieces of evidence multiply in odds, so they add in log-odds. Naive Bayes is literally a sum of logits.

- **IRT is linear in the logit.** $a_j(\theta_s - b_j)$ is a straight line in ability. Difficulty shifts it; discrimination scales it.

INTUITION

A logit is a "score" in a game with no upper limit. Zero means a coin flip; $+2$ means roughly 88% — confident but not certain; $+6$ means 99.75%. Adding one unit of logit multiplies the odds by $e \approx 2.72$. This is why a model can be "confident" at logit 4 and still be wrong: 98% is not 100%.

SOLVED IN 60 SECONDS

Take $p = 0.8$. Odds $= 0.8/0.2 = 4$; logit $= \log 4 = 1.386$.

Now subtract 0.7 of evidence: $z = 1.386 - 0.7 = 0.686$, so $p = \sigma(0.686) = 0.665$.

Check the derivative at $z = 0$: $\sigma'(0) = 0.25$. A small step $\Delta z = 0.1$ changes p by about 0.025 near the middle, but only by about 0.002 at $z = 4$.

IN THIS REPO

Every model outputs a logit and calls `sigmoid` once at the end; calibration (`src/aims_starter/calibration.py`) operates on that logit, not on the probability. Temperature scaling divides the logit by T — a one-line change in logit space that is messy in probability space.

3 Likelihood and log loss

A likelihood asks "how plausible are these labels under this probability?", and its negative log is the loss the network minimizes.

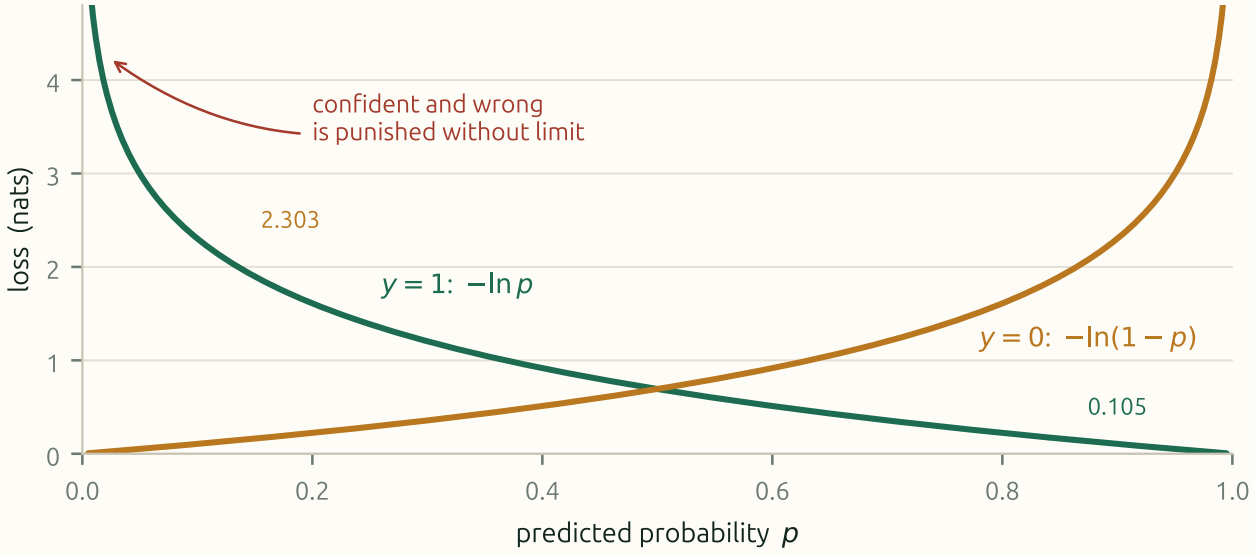
For one binary outcome, the probability of what actually happened can be written in one line:

$$P(y \mid p) = p^{y}(1-p)^{1-y} = \begin{cases} p, & y = 1, \\ 1-p, & y = 0. \end{cases}$$

For n independent outcomes, the likelihood is the product $\prod_i p_i^{y_i}(1-p_i)^{1-y_i}$. Products underflow and have awkward derivatives, so we take the log. Maximizing the log-likelihood is the same as minimizing the **negative log-likelihood** (NLL):

$$\mathrm{NLL}(p) = -\frac{1}{n}\sum_{i=1}^{n} [y_i \log p_i + (1-y_i)\log(1-p_i)].$$

This is exactly binary cross-entropy. With a sigmoid, PyTorch's `binary_cross_entropy_with_logits` computes it stably in one step.



The loss is asymmetric in a useful way. Predicting 0.9 when $y = 1$ costs 0.105 nats; predicting 0.1 when $y = 1$ costs 2.303. Being confidently wrong is punished without bound, while being uncertain is cheap.

THE MLE OF A BERNOULLI IS JUST THE BASE RATE

Set the derivative of the total log-likelihood to zero: $\sum_i (y_i - p)/[p(1-p)] = 0$, so $\hat{p} = \frac{1}{n}\sum_i y_i$. With no prior, the best constant probability is the observed frequency. That is the "empirical mean" baseline, and it is already hard to beat when there is no item information.

INTUITION

Log loss is a penalty measured in "surprise". If you say an event has probability 0.9 and it happens, you are only mildly surprised ($-\log 0.9 = 0.105$). If you say 0.1 and it happens, you are very surprised ($-\log 0.1 = 2.303$). The average surprise is the loss. A model that is never surprised is either perfect or cheating.

SOLVED IN 60 SECONDS

Three outcomes $y = (1, 0, 1)$.

Predict $p = 0.7$ everywhere: $\text{NLL} = -[2\log 0.7 + \log 0.3]/3 = (0.713 + 1.204)/3 = 0.639$.

Predict $p = 0.9$: $\text{NLL} = -[2\log 0.9 + \log 0.1]/3 = (0.211 + 2.303)/3 = 0.838$.

The base rate is $2/3 = 0.667$, with NLL 0.6365. So 0.7 is close to optimal; 0.9 is much worse. **More confidence is not more skill.**

TRAP

Cross-entropy is not calibrated by construction. A network can drive NLL down by becoming confident on easy training items and still output badly scaled probabilities on a new benchmark. That is a distribution-shift problem, not an optimization bug — see §8 and §18.

4 Bayes, priors, and shrinkage

A prior is a small, honest lie that keeps an estimate sane when the data are scarce — and the posterior is the corrected version of that lie.

Bayes' rule for parameters π and observed labels y:

$$\underbrace{p(\pi \mid y)}_{\text{posterior}} \propto \underbrace{p(y \mid \pi)}_{\text{likelihood}}\,\underbrace{p(\pi)}_{\text{prior}}.$$

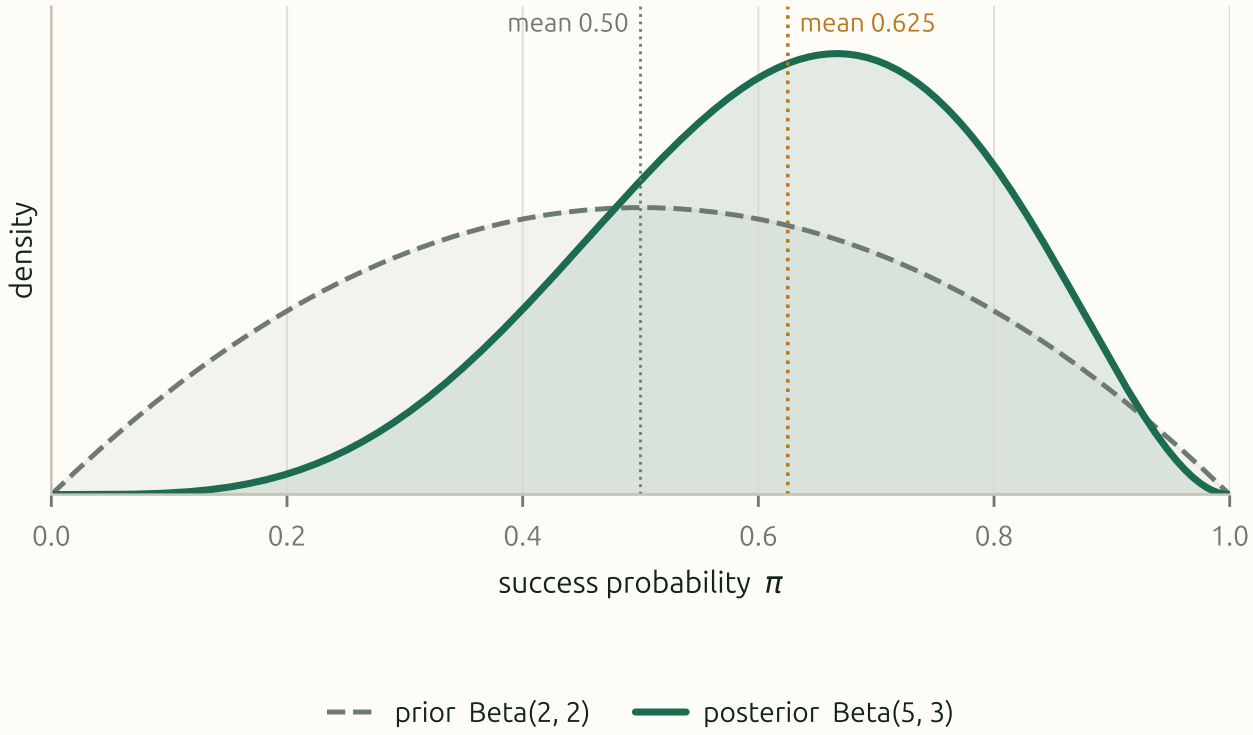
The missing denominator is a constant with respect to π, so it does not affect the shape or the argmax. With a Bernoulli likelihood and a Beta prior, the posterior is Beta again — **conjugacy**:

$$\pi \sim \text{Beta}(\alpha, \beta), \quad y_i \sim \text{Bernoulli}(\pi) \implies \pi \mid y \sim \text{Beta}\left(\alpha + \sum_i y_i,\ \beta + n - \sum_i y_i\right).$$

The posterior mean is a weighted average of the prior mean and the data mean:

$$\mathbb{E}[\pi \mid y] = \frac{\alpha + k}{\alpha + \beta + n} = \frac{\alpha + \beta}{\alpha + \beta + n}\,\frac{\alpha}{\alpha + \beta} + \frac{n}{\alpha + \beta + n}\,\frac{k}{n}.$$

The prior acts like $\alpha + \beta$ pseudo-observations. That is the entire idea of **shrinkage**: with little data, trust the prior; with a lot, trust the data.



Three successes in four trials, starting from Beta(2, 2). The posterior is Beta(5, 3); its mean moves from 0.50 to 0.625, not all the way to the data mean 0.75. The prior is still pulling.

INTUITION

You have seen a coin land heads once. Is $P(\text{heads}) = 1$? Common sense says no. A prior of Beta(2, 2) says "start at half, with the weight of four imaginary flips", so one head gives $3/5 = 0.6$, not 1.0. The prior is not a claim about the world; it is a variance-control device. Its strength is a dial you can defend and ablate.

SOLVED IN 60 SECONDS

Prior $\text{Beta}(2, 2)$; observe $k = 3$ successes in $n = 4$ trials.

Posterior: $\text{Beta}(2 + 3,\ 2 + 1) = \text{Beta}(5, 3)$.

Posterior mean: $5/8 = 0.625$. Posterior mode: $(5 - 1)/(8 - 2) = 4/6 = 0.667$. Data mean: 0.75. Prior mean: 0.5.

The posterior mean sits between prior and data, closer to the prior because the prior has weight 4 and the data have weight 4 — hence exactly halfway between 0.5 and 0.75.

IN THIS REPO

The Beta-Binomial control uses $p = (2 + \sum y_i)/(4 + n)$, i.e. $\text{Beta}(2, 2)$ updated online. It is deliberately simple: any neural model that cannot beat it at low budgets is not earning its complexity. The MAP ability fit in `adaptation.py` is the same idea in logit space, with prior $\mathcal{N}(0, 4)$.

5 Uncertainty, paired tests, and honest error bars

A difference of 0.003 in Brier ALC is not a result until you know how much the number would move if you reran the experiment.

An estimator has a **standard error**: the standard deviation of its sampling distribution. For a mean $\bar{x}$ of n independent observations, $\text{SE} = \hat{\sigma}/\sqrt{n}$. A 95% interval is roughly $\bar{x} \pm 1.96\,\text{SE}$ for large n, and wider for small n (use the t distribution).

But evaluation data are not independent items. Items cluster inside benchmarks, and repeated trials cluster inside subject–item pairs. The right unit of resampling is the **cluster** — usually the subject–benchmark task. Two consequences:

- Average within a task first, then across tasks (macro-average). Do not let a benchmark with 5,000 items outvote one with 50.
- Bootstrap *tasks*, not rows, to get intervals that reflect benchmark-to-benchmark variation.

For comparing two models, use **paired** differences: compute $d_t = \text{Brier}_A(t) - \text{Brier}_B(t)$ on the same task t. The shared difficulty of the task cancels, and the standard error of $\bar{d}$ is usually much smaller than the standard error of either score alone.

SOLVED IN 60 SECONDS

Five tasks, Brier differences $A - B = (-0.02, -0.01, -0.02, -0.01, -0.01)$.

Mean $= -0.014$. Sample SD $= 0.00548$. SE $= 0.00548/\sqrt{5} = 0.00245$.

$t = -0.014/0.00245 = -5.72$ on 4 degrees of freedom. A 95% interval is about $-0.014 \pm 2.78(0.00245) = [-0.021, -0.007]$.

Read this: the effect is consistent in sign, but five tasks is five tasks. Report the interval, not just the p-value, and do not extrapolate to "all benchmarks".

TRAP

Treating 10,000 item predictions as 10,000 independent samples gives absurdly narrow error bars. The effective sample size is closer to the number of independent *tasks*, which may be 20. This is the most common statistical mistake in benchmark papers.

6 Entropy, KL, and the value of a label

Entropy measures how unsure you are; KL measures how wrong your distribution is; their difference tells you what one more label could buy.

For a Bernoulli outcome with success probability p, the entropy in nats is

$$H(p) = -p \log p - (1-p) \log(1-p).$$

It is 0 at $p = 0$ and $p = 1$, and maximal at $p = 0.5$ with $H = 0.693$ nats. Entropy is the average surprise of the *true* distribution; cross-entropy is the average surprise when you predict q but the truth is p:

$$H(p, q) = -\sum_y p(y) \log q(y) = H(p) + \text{KL}(p \,\|\, q).$$

So minimizing log loss is minimizing cross-entropy, and the extra cost over the irreducible entropy is the KL divergence. KL is non-negative, zero only when the distributions match, and **asymmetric**: $\text{KL}(p\|q) \neq \text{KL}(q\|p)$ in general.

For two univariate Gaussians, the formula you will see in the NP objective is

$$\text{KL}\big(\mathcal{N}(\mu_1, \sigma_1^2) \,\|\, \mathcal{N}(\mu_2, \sigma_2^2)\big) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}.$$

INTUITION

Entropy is how wide your uncertainty is. KL is the price of betting on the wrong distribution. Information gain is the amount by which a label is expected to narrow the width. An item that is always answered correctly by everyone carries almost no information: its outcome is already predictable. An item near 50/50 carries one full bit. This is why acquisition policies prefer uncertain items — but only if the uncertainty is real, not ignorance.

SOLVED IN 60 SECONDS

$H(0.9) = -0.9 \log 0.9 - 0.1 \log 0.1 = 0.105 + 0.230 = 0.325$ nats.

$\text{KL}(\text{Bern}(0.9) \,\|\, \text{Bern}(0.5)) = 0.9 \log(0.9/0.5) + 0.1 \log(0.1/0.5) = 0.529 - 0.161 = 0.368$ nats.

If a label moves your belief from 0.5 to 0.8, the entropy drops from 0.693 to 0.500, an information gain of 0.193 nats.

IN THIS REPO

The acquisition ablations compare random, entropy, and Fisher-information orderings. Under 1PL, entropy-greedy and Fisher-greedy rank items identically; under 2PL they diverge only through a_j^2. The predictor is held fixed while the order changes, so the experiment isolates the value of the label stream, not the model.

PART II

Scoring a probabilistic predictor

The competition pays in Brier score, averaged over budgets. Calibration and discrimination are the two ways that score can fail, and they need different fixes.

- 7 · Brier score
- 8 · Calibration
- 9 · Discrimination and AUC
- 10 · The learning curve and ALC
- 11 · Who gets weight?

7 Brier score: the only number you are paid on

Brier is mean squared error applied to probabilities. It rewards being right and honest at the same time.

For n predictions p_i and outcomes $y_i \in \{0, 1\}$:

$$\mathrm{BS} = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i)^2.$$

Each term is small when the probability is close to the outcome. Predicting 0.9 on a success costs 0.01; predicting 0.9 on a failure costs 0.81. The score is in squared-probability units, bounded in $[0, 1]$, lower is better.

IT IS A PROPER SCORING RULE

Suppose the true success probability is π. The expected Brier score of a forecast p is

$$\mathbb{E}[(p - Y)^2] = (p - \pi)^2 + \pi(1 - \pi).$$

The second term does not depend on p; the first is minimized uniquely at $p = \pi$. So the best expected score is achieved by reporting your true belief. No other probability can do better in expectation. This property — **strict propriety** — is why the competition can use Brier without worrying that competitors will game it by scattering predictions.

THE DECOMPOSITION WORTH REMEMBERING

Group predictions into bins k with mean forecast $\bar{p}_k$ and observed frequency $\bar{y}_k$:

$$\mathrm{BS} = \underbrace{\frac{1}{n} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2}_{\text{reliability}} - \underbrace{\frac{1}{n} \sum_k n_k (\bar{y}_k - \bar{y})^2}_{\text{resolution}} + \underbrace{\bar{y}(1 - \bar{y})}_{\text{uncertainty}}.$$

- **Reliability**: how far bin averages are from observed frequencies. Calibration error. Lower is better.
- **Resolution**: how much the bins differ from the overall base rate. Higher is better — it is the model's ability to separate easy from hard cases.
- **Uncertainty**: the irreducible variance of the outcomes. It depends on the dataset, not the model.

> **INTUITION**
>
> Brier rewards two different skills. **Resolution** is "can you tell which items are harder?". **Reliability** is "when you say 70%, is it really 70%?". A model can be excellent at ranking items and still get a bad Brier score if its probabilities are on the wrong scale. That is not a contradiction; it is the most common failure in this repo.

> **SOLVED IN 60 SECONDS**
>
> $y = (1, 0, 1, 0)$, $p = (0.9, 0.2, 0.6, 0.4)$.
>
> Squared errors: 0.01, 0.04, 0.16, 0.16. Brier $= 0.37/4 = 0.0925$.
>
> Constant-0.5 baseline: 0.25. Skill score $= 1 - 0.0925/0.25 = 0.63$, i.e. 63% of the baseline's error removed.
>
> Now move one prediction from 0.9 to 0.99 on the success: error falls from 0.01 to 0.0001 — but move it to 0.99 on a failure and the error explodes from 0.81 to 0.9801. Asymmetric risk in one line.

TRAP

Do not compare raw Brier scores across benchmarks with different base rates. The floor is $\bar{y}(1-\bar{y})$, not zero: a benchmark with 10% successes has a much lower floor than one with 50%. Use a skill score, or compare only on identical tasks.

8 Calibration: are your probabilities honest?

Calibration is the agreement between the number you say and the frequency that happens. It is separate from being able to tell items apart.

Among all cases where you predicted about 0.7, roughly 70% should succeed. If only 50% succeed, you are **overconfident**. If 85% succeed, you are **underconfident**. A reliability diagram plots observed frequency against predicted probability; perfect calibration is the diagonal.

The scalar summary used here is the **expected calibration error** over K equal-width bins:

$$\mathrm{ECE} = \sum_{k=1}^{K} \frac{n_k}{n} \left| \bar{p}_k - \bar{y}_k \right|.$$

ECE is a diagnostic, not the competition metric. It depends on the number of bins, it ignores ordering within a bin, and it is not a proper scoring rule — you can reduce ECE by throwing away resolution. Report it alongside Brier, never instead of it.

THE TWO STANDARD FIXES

- **Temperature scaling.** $p' = \sigma(z/T)$. One parameter, fitted on held-out data. $T > 1$ softens overconfidence; $T < 1$ sharpens underconfidence. It is monotone, so AUC does not change.
- **Platt scaling.** $p' = \sigma(az + b)$. Two parameters; can fix both scale and bias. Again monotone if $a > 0$.

Neither fix creates resolution. If the model cannot tell easy items from hard ones, no monotone rescaling will help; it only moves the probability scale.

INTUITION

Imagine a weather forecaster who says “70% chance of rain” on 100 days, and it rains on 70 of them. Calibration is that match. Now imagine a forecaster who always says 70% and one who says 20% on dry days and 90% on wet days. Both can be calibrated; the second is more useful. Brier rewards both calibration and usefulness; ECE only checks the first.

SOLVED IN 60 SECONDS

Two bins of five predictions each.

Bin 1: mean prediction 0.84, observed 3/5 = 0.60. Gap 0.24.

Bin 2: mean prediction 0.44, observed 3/5 = 0.60. Gap 0.16.

$\mathrm{ECE} = 0.5(0.24) + 0.5(0.16) = 0.20$. The model is overconfident in the first bin and underconfident in the second. A single temperature may not fix both; that is a signal of a shape problem, not just a scale problem.

Apply $T = 2$ to logits $(-1, 0, 1)$: they become $(-0.5, 0, 0.5)$, probabilities $(0.378, 0.5, 0.622)$. Ordering is preserved; only the spread changes.

IN THIS REPO

`AnchorCalibrator` fits a scale, a bias, and a weight on the context base rate, separately per budget, on meta-calibration folds from training benchmarks. The context-rate anchor is the only task-specific quantity available at low budgets, so it is blended in rather than trusted outright.

9 Discrimination and AUC: can you tell items apart?

AUC measures ranking, not probability. It answers “is a random success scored above a random failure?”, and it is blind to whether the scores are calibrated.

For every positive–negative pair, count a win if the positive has the higher score, a tie as half. That fraction is the area under the ROC curve:

$$\mathrm{AUC} = P\big(s_+ > s_-\big) + \tfrac{1}{2} P\big(s_+ = s_-\big).$$

0.5 is chance, 1.0 is perfect ordering. AUC is invariant to any strictly increasing transformation of the scores: adding 3, multiplying by 10, or applying a sigmoid all leave it unchanged. It is therefore the right diagnostic for "does the model have signal?" and the wrong metric for "are the probabilities usable?".

INTUITION

AUC is a *ranking* score, like a leaderboard that only cares about order. Brier is a *probability* score, like a weather forecast that must state the number. A model can rank perfectly and still say "99%" for everything — AUC 1.0, terrible Brier. Conversely, a model can be perfectly calibrated at the base rate and have AUC 0.5: it says 30% to everyone, and 30% is right on average, but it cannot distinguish anything.

SOLVED IN 60 SECONDS

Scores $s = (0.9, 0.8, 0.4, 0.3)$, outcomes $y = (1, 0, 1, 0)$. Positives: 0.9, 0.4. Negatives: 0.8, 0.3.

Pairs: $0.9 > 0.8$ ✓, $0.9 > 0.3$ ✓, $0.4 > 0.8$ ✗, $0.4 > 0.3$ ✓. Three wins out of four: $\mathrm{AUC} = 0.75$.

Now transform every score by $s \mapsto 10s + 2$: the same three pairs win, AUC stays 0.75. The Brier score, however, changes dramatically because the probabilities are no longer in $[0, 1]$.

TRAP

"AUC went up" is not evidence that the submission improved. The leaderboard is Brier ALC. Optimize a proper score; use AUC only to diagnose whether the model has resolution.

10 The learning curve and Brier ALC

The competition does not score one budget. It scores the whole path from zero labels to thirty-one, with the middle of the path weighted most.

At each budget $B \in \{0, 1, 3, 7, 15, 31\}$, reveal the first B outcomes of the acquisition pool (the prefixes are nested), recompute predictions for the untouched evaluation pool, and compute Brier. That gives six numbers $B_0, \dots, B_{31}$. The official aggregate is

$$\mathrm{ALC} = 0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}.$$

The weights sum to one, so ALC is on the same scale as Brier. The endpoints get half weight. That is deliberate: budget 0 is almost pure prior, budget 31 is almost pure interpolation, and the interesting adaptation happens in between. A model that wins only at $B = 31$ but loses at $B = 0$ can easily lose overall.

The curve need not be monotone. A single noisy label at $B = 1$ can make predictions worse; a model that overreacts to the first label can produce a visible bump. That bump is information, not necessarily a bug — but it must be explained, not hidden.

INTUITION

ALC is a weighted average of six exam grades, with the middle four counting double. It rewards a model that is good *early* and stays good, not one that memorizes the final context. If you only tune the last budget, you are optimizing 10% of the score.

SOLVED IN 60 SECONDS

Model Brier: $(0.250, 0.243, 0.231, 0.218, 0.205, 0.199)$.

$\mathrm{ALC} = 0.1(0.250) + 0.2(0.243) + 0.2(0.231) + 0.2(0.218) + 0.2(0.205) + 0.1(0.199)$

$= 0.0250 + 0.0486 + 0.0462 + 0.0436 + 0.0410 + 0.0199 = 0.2243.$

Constant-0.5 baseline: 0.250. Relative gain: $(0.250 - 0.2243)/0.250 = 10.3\%$.

Notice that the $B = 0$ term contributes only 0.025, while the four middle terms contribute 0.179 — 80% of the score.

TRAP

Computing ALC from budgets that are not nested, or from different acquisition orders per budget, silently changes the experiment. "Budget 7" must contain the labels revealed at budgets 1 and 3; otherwise the curve is not a learning curve.

11 Who gets weight? Aggregation and the leaderboard

The same predictions can score 0.218 or 0.300 depending on whether you average items or tasks. The competition averages tasks.

Three levels are easy to confuse:

- **Item level:** one squared error per evaluation item. A task with 5,000 items dominates one with 50.
- **Task level:** average items within a subject–benchmark pair, then average pairs equally. This is the leaderboard's unit.
- **Benchmark level:** average tasks within a benchmark, then benchmarks. Useful for reporting, but not the official weighting unless the organizer says so.

Micro-averaging pools all items; macro-averaging pools tasks. They answer different questions. Micro asks "how good is the average prediction?"; macro asks "how good is the average task?". When task sizes differ, the two can disagree in sign as well as magnitude.

SOLVED IN 60 SECONDS

Task A: 100 items, Brier 0.20. Task B: 10 items, Brier 0.40.

Micro $= (100 \cdot 0.20 + 10 \cdot 0.40)/110 = 24/110 = 0.218$.

Macro $= (0.20 + 0.40)/2 = 0.300$.

A 0.082 gap — larger than most model improvements in this competition — produced purely by the averaging choice. Always state which one you report.

TRAP

If you compute a confidence interval over items, a big benchmark will dominate and the interval will look tiny. Resample *tasks* (or benchmarks) instead. The official protocol averages each eligible subject–benchmark pair equally before the leaderboard average.

METRIC	WHAT IT MEASURES	PROPER?	USE IT FOR
Brier	squared probability error	yes, strictly	the competition; model selection
Log loss	negative log-likelihood	yes, strictly	training; sharper penalty for confidence
AUC	ranking	no	diagnosing resolution only
ECE	bin-wise calibration	no	diagnosing probability scale
Accuracy	thresholded decisions	no	almost never, here

PART III

Item response theory

IRT is the skeleton of the whole competition: one number for the subject, two numbers per item, and a logistic link. Amortized IRT replaces the per-item table with a network that reads item content.

- 12 · The 2PL model
- 13 · Fitting ability with MAP
- 14 · Amortized IRT
- 15 · Identifiability and leakage

12 The 2PL model: ability meets difficulty

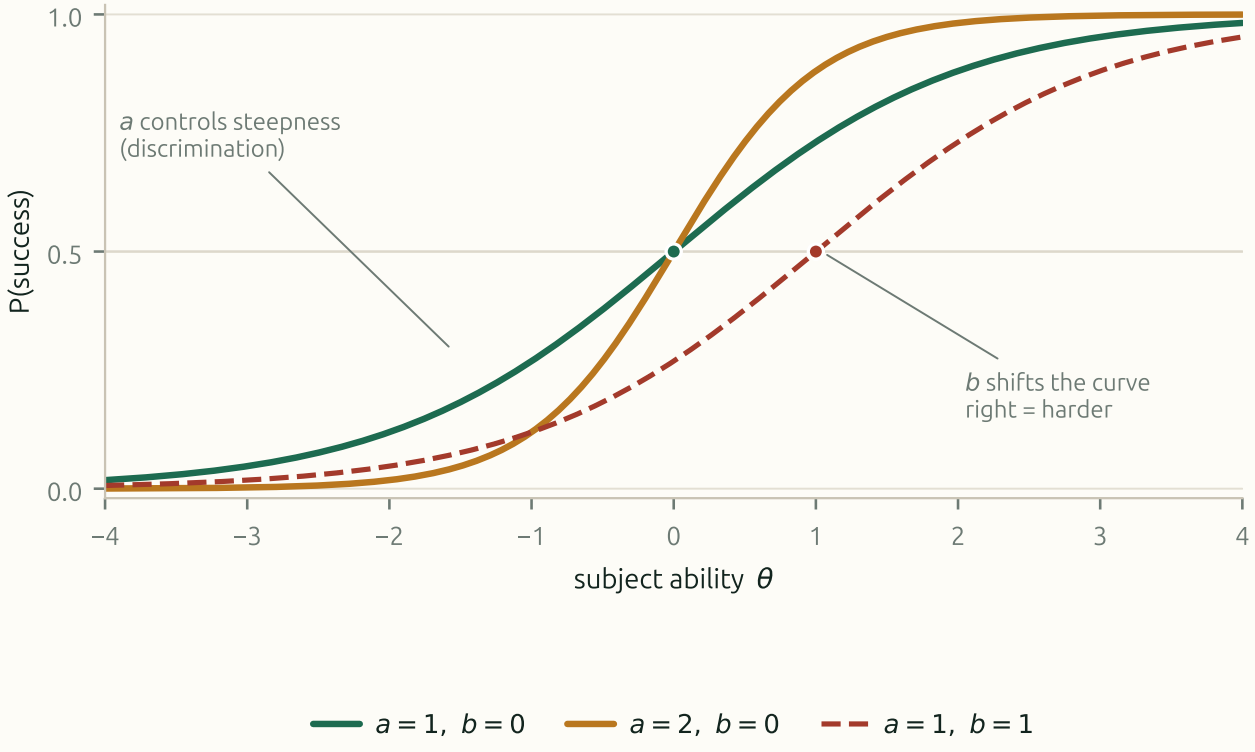
Every prediction in this repo is a logistic comparison between how able the subject is and how hard the item is, optionally sharpened by how well the item discriminates.

For subject s and item j, the two-parameter logistic model is

$$p_{sj} = P(y_{sj} = 1) = \sigma\big(a_j(\theta_s - b_j)\big),$$

with three interpretable quantities:

- $\theta_s \in \mathbb{R}$: the subject's **ability**. Higher is better. It is a position on the same scale as difficulty.
- $b_j \in \mathbb{R}$: the item's **difficulty**. The curve crosses 0.5 at $\theta_s = b_j$; a harder item sits to the right.
- $a_j > 0$: the item's **discrimination**. It controls steepness. A large a_j means the item separates nearby ability levels sharply; $a_j \to 0$ makes it a coin flip.



Item characteristic curves. Increasing b shifts the curve right (harder at every ability). Increasing a makes it steeper (more informative near the threshold). The dashed curve is the same a as the green one, just harder.

The relatives are worth knowing. **1PL** (Rasch) sets every $a_j = 1$ and learns only difficulties. **3PL** adds a guessing floor c_j:

$$p_{sj} = c_j + (1 - c_j)\,\sigma\big(a_j(\theta_s - b_j)\big).$$

This repo uses 2PL only. There is no guessing mechanism on free-form harm labels, and (a, b, c) is not identifiable from 31 labels. That is a modelling decision, not an omission.

INTUITION

Picture a ruler. Ability and difficulty are both measured on it. An item is a hurdle at position b_j; a subject at θ_s clears it with probability given by the gap $\theta_s - b_j$. Discrimination a_j is how sharply the hurdle decides: a low-discrimination item is a fuzzy hurdle that lets almost anyone through with middling probability; a high-discrimination item is a razor edge.

SOLVED IN 60 SECONDS

Take $\theta = 1$, $b = 0.5$, $a = 1.5$: logit $= 1.5(0.5) = 0.75$, so $p = \sigma(0.75) = 0.679$.

Make the item harder, $b = 1.5$: logit $= 1.5(-0.5) = -0.75$, $p = 0.321$. Same subject, different hurdle.

Keep $b = 0.5$ but lower discrimination to $a = 0.5$: logit $= 0.25$, $p = 0.562$. Raise it to $a = 3$: logit $= 1.5$, $p = 0.818$.

Read this: difficulty moves the midpoint; discrimination changes how fast the curve rises around it.

TRAP

The sign convention is easy to flip. Here, larger b means *harder* because it is subtracted. If you see $a(\theta + b)$ somewhere, difficulty has the opposite sign. Check the formula before comparing numbers across papers.

13 Fitting ability: MAP and Newton in five lines

With item parameters fixed, estimating a subject's ability is a one-dimensional concave problem — which is why the repo can afford an exact Newton solve inside a neural model.

Given acquired labels $C = \{(a_i, b_i, y_i)\}$, the penalized log-posterior for θ is

$$\log p(\theta \mid C) = \sum_{i \in C} \Big[y_i \log \sigma\big(a_i(\theta - b_i)\big) + (1 - y_i) \log\big(1 - \sigma(a_i(\theta - b_i))\big) \Big] - \frac{(\theta - \mu_0)^2}{2\sigma_0^2} + \text{const.}$$

Its first two derivatives are clean:

$$g(\theta) = \sum_{i \in C} a_i (y_i - p_i) - \frac{\theta - \mu_0}{\sigma_0^2}, \qquad h(\theta) = -\sum_{i \in C} a_i^2 \, p_i (1 - p_i) - \frac{1}{\sigma_0^2}.$$

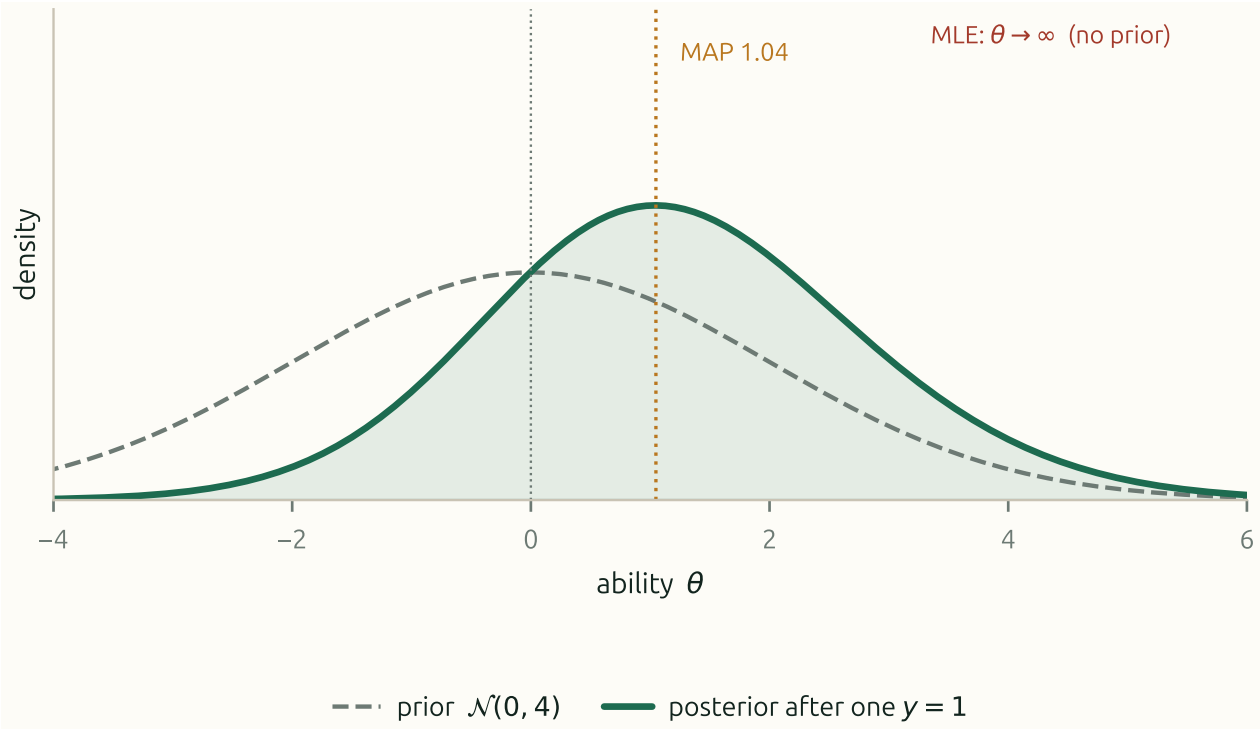
Newton's method jumps to the top of the local quadratic:

$$\theta \leftarrow \theta - \frac{g(\theta)}{h(\theta)}.$$

Because the logistic log-likelihood is concave in θ and the prior is Gaussian, the posterior is log-concave: there is one optimum, and Newton converges quickly. The repo runs 12 steps, clamps to $[-4, 4]$, and uses $\mu_0 = 0, \sigma_0^2 = 4$.

INTUITION

Without a prior, one success on one item is "proof" that the subject never fails, so the maximum-likelihood ability runs off to infinity. The prior says "abilities are usually within a couple of units of zero", and pulls the estimate back. This is not a hack; it is the only way to get a finite answer from separated data. The price is a little bias, paid gladly for a lot of variance reduction.



One success is not infinite ability. The prior keeps the estimate at 1.04 instead of running to infinity. The posterior is the prior multiplied by the likelihood, then renormalized; the MAP is its peak.

SOLVED IN 60 SECONDS

One item, $a = 1$, $b = 0$, observed $y = 1$. Prior $\mathcal{N}(0, 4)$, start $\theta_0 = 0$.

At $\theta = 0$: $p = 0.5$, $g = 1(0.5) - 0 = 0.5$, $h = -1(0.25) - 0.25 = -0.5$. Step $= -0.5/(-0.5) = 1$, so $\theta = 1$.

At $\theta = 1$: $p = 0.731$, $g = 0.269 - 0.25 = 0.019$, $h = -0.197 - 0.25 = -0.447$. Step $= 0.042$, so $\theta = 1.042$.

Next step: 1.0426. The MLE, with no prior, would keep going: $\theta \to \infty$.

IN THIS REPO

`map_ability` in `src/aims_starter/adaptation.py` is exactly this update, vectorized over context items. The neural models never output θ_s; they output item parameters and corrections, and the analytic MAP owns the ability level. That keeps the latent variable interpretable and prevents the network from hiding the level in an unconstrained scalar.

14 Amortized IRT: difficulty from content, not a lookup table

Classical IRT stores one difficulty per item. Amortized IRT learns a function from item content to difficulty, so an unseen item gets a sensible prior instead of a blank.

The item network f_ϕ reads the item input x_j (text embedding, metadata, interactors) and outputs the 2PL parameters:

$$(\tilde{a}_j,\ b_j) = f_\phi(x_j), \qquad a_j = \mathrm{softplus}(\tilde{a}_j) + \epsilon.$$

The softplus keeps discrimination positive; ϵ is a small floor for numerical stability. Ability is then fitted by the MAP of §13, and the prediction is $\sigma(a_j(\theta_s - b_j))$. Training minimizes binary cross-entropy over observed cells of the response matrix. Gradients flow into f_ϕ, not into a per-item table.

WHY "AMORTIZED" IS THE IMPORTANT WORD

Amortization means paying a one-time cost to learn an inference function, then reusing it. Here the "inference" is item-parameter prediction: instead of fitting b_j for every new item at test time, the network predicts it in one forward pass. This is what makes cold-start evaluation possible. The alternative — fit a new IRT model per benchmark — cannot score items whose outcomes were never observed.

INTUITION

A human test designer can guess that a long, multi-step coding task is harder than a one-line syntax question, before seeing anyone attempt either. Amortized IRT learns exactly that guess from data: item text goes in, a difficulty and a discrimination come out. The subject's ability then calibrates the guess against the few labels you have.

SOLVED IN 60 SECONDS

Toy item network: $b = 1.2x - 0.3$, $a = \text{softplus}(0.4)$. Item $x = 0.5$, so $b = 0.3$, $a = \log(1 + e^{0.4}) = 0.913$. Subject $\theta = 0.6$.

Logit $= 0.913(0.6 - 0.3) = 0.274$; $p = \sigma(0.274) = 0.568$.

If the observed outcome is $y = 1$, the BCE gradient with respect to the logit is $p - y = -0.432$. Through the item net: $\partial L/\partial b = a(y - p) = 0.913(0.432) = 0.394$, and $\partial L/\partial w = 0.394 \cdot x = 0.197$.

Gradient descent lowers w, which lowers b, which raises the predicted probability. The chain rule is doing all the work.

IN THIS REPO

The item encoder reads a 1,157-dimensional vector: 384-d MiniLM item embedding, five numeric item features, 384-d subject embedding, 384-d benchmark embedding. The subject slice is **masked** on the item-parameter path, so a_j, b_j cannot become subject-conditioned. Without that mask, "item difficulty" silently becomes "difficulty for this subject", and the IRT interpretation collapses.

TRAP

Amortization does not eliminate the need for held-out *items*. If the same item appears in training and test, the network can memorize its difficulty through the embedding, and the cold-start claim is false. Split by item (and by benchmark), not by random response rows.

15 Identifiability, conventions, and leakage

IRT is identifiable only up to a shift and a scale, and the competition's real risk is not math — it is information that quietly crosses the train/test boundary.

THE SHIFT AND SCALE INVARIANCES

For any constants c and $k > 0$, the 1PL model is unchanged by

$$\theta_s \mapsto \theta_s + c, \quad b_j \mapsto b_j + c, \qquad \text{or} \qquad \theta_s \mapsto k\theta_s, \quad b_j \mapsto kb_j, \quad a_j \mapsto a_j/k.$$

Only *relative* positions are identified. Fix a convention: θ has mean zero and a fixed prior variance, and $a_j > 0$. The repo uses $\theta \sim \mathcal{N}(0, 4)$ and clamps to $[-4, 4]$. Without a convention, two fits can look different but predict identically.

LEAKAGE IS BROADER THAN "USING THE TEST LABELS"

- **Direct:** target outcomes enter the posterior, the calibration fit, or early stopping.
- **Identity:** a subject or benchmark embedding lets the item path memorize that task's base rate.
- **Duplicates:** the same item (or a paraphrase) appears in train and test; the embedding is a fingerprint.
- **Order:** the acquisition order was chosen using evaluation outcomes, even if the labels themselves are legal.
- **Selection:** the checkpoint was chosen by its test ALC. That is leakage at the experiment level, not the code level.

SOLVED IN 60 SECONDS

Two parameter sets for the same item:

$\theta = 1, b = 0, a = 1$: logit $= 1(1 - 0) = 1$.

$\theta = 2, b = 1, a = 1$: logit $= 1(2 - 1) = 1$. Same probability, different "ability" and "difficulty".

Scale: $a = 2, \theta = 0.5, b = 0$ gives logit 1; $a = 1, \theta = 1, b = 0$ also gives logit 1. The data cannot distinguish them without a convention.

INTUITION

Identifiability is "can the data tell these two stories apart?". If every prediction is identical under both, the answer is no, and any claim about the absolute value of θ is a claim about your convention, not about the world. This is why papers standardize ability and why the repo fixes a prior instead of reporting a raw MLE.

TRAP

At 31 labels, a 3PL guessing parameter is not identified: many (a, b, c) triples fit the same six or seven outcomes equally well. The repo skips 3PL for this reason. Adding parameters that the data cannot constrain does not make the model more scientific; it makes the error bars dishonest.

PART IV

Adaptation with almost no labels

Thirty-one labels is not a training set; it is a handful of hints. The methods that work here are the ones that know how little they know: conjugate updates, partial pooling, and calibration fitted somewhere else.

- 16 · The Beta-Binomial control
- 17 · Hierarchical shrinkage
- 18 · Calibration in practice

16 The Beta-Binomial control that is hard to beat

Before any neural network, there is a one-line predictor that updates a single probability with every revealed label. It is the bar every model must clear.

Start with $\pi \sim \mathrm{Beta}(2, 2)$, whose mean is 0.5. After n labels with k successes, the posterior is $\mathrm{Beta}(2 + k,\, 2 + n - k)$, and the predictive probability for the next item is the posterior mean:

$$\hat{p}_n = \frac{2 + k}{4 + n}.$$

This is exactly the formula in the repo. It ignores item content entirely: every evaluation item gets the same probability. That sounds naive, and it is — but it is also robust, impossible to overfit, and surprisingly strong when the item signal is weak or the benchmark is new.

INTUITION

The Beta-Binomial is a thermostat. It starts at 50%, and each label nudges the setting. One success moves it to 60%; three successes in seven moves it to 45% if failures dominate. It never panics, never claims certainty, and never needs a GPU. A neural model that loses to it at $B = 0$ or $B = 1$ is not adding value where value is scarcest.

SOLVED IN 60 SECONDS

Labels arrive in order $1, 0, 1, \ldots$.

$B = 0$: $\hat{p} = 2/4 = 0.500$.

$B = 1$ (one success): $\hat{p} = 3/5 = 0.600$.

$B = 3$ (two successes, one failure): $\hat{p} = 4/7 = 0.571$.

$B = 7$ (three successes, four failures): $\hat{p} = 5/11 = 0.455$.

Notice that adding labels can move the prediction either way. “More data” does not guarantee a better Brier score on a single task; it guarantees a better estimate of the task’s base rate.

TRAP

The control is not a straw man to be dismissed in the related-work section. If your model’s ALC is 0.27 and the control is 0.25, the correct conclusion is that your model is worse than a constant, not that the control is “too simple to compare against”.

17 Hierarchical shrinkage and empirical Bayes

When many small groups each have a noisy estimate, the best estimate for each group is usually a compromise between its own data and the pooled data.

Suppose task g has n_g labels and k_g successes. A hierarchical model gives each task its own rate π_g, but lets those rates come from a common Beta prior:

$$\pi_g \sim \mathrm{Beta}(\kappa\mu,\ \kappa(1-\mu)), \qquad k_g \mid \pi_g \sim \mathrm{Binomial}(n_g, \pi_g).$$

The posterior mean for task g is a shrinkage estimate:

$$\hat{\pi}_g = \frac{k_g + \kappa\mu}{n_g + \kappa} = \underbrace{\frac{n_g}{n_g + \kappa}}_{\text{trust the data}} \cdot \frac{k_g}{n_g} + \underbrace{\frac{\kappa}{n_g + \kappa}}_{\text{trust the pool}} \cdot \mu.$$

μ is the pooled rate and κ is the prior strength in pseudo-observations. **Empirical Bayes** estimates μ and κ from the training tasks instead of fixing them by hand. With $\kappa = 0$ you get separate per-task rates; with $\kappa \to \infty$ you get one pooled rate. Partial pooling is the interesting middle.

SOLVED IN 60 SECONDS

Task A: $1/1$. Task B: $3/9$. Pooled rate $\mu = 4/10 = 0.4$. Choose $\kappa = 4$.

Separate: 1.00 and 0.333. Pooled: 0.400 for both.

Shrunk: A $= (1 + 4 \cdot 0.4)/(1 + 4) = 2.6/5 = 0.520$; B $= (3 + 1.6)/(9 + 4) = 4.6/13 = 0.354$.

Task A's perfect record is pulled down hard because one observation is weak evidence; Task B barely moves because nine observations already pin it down. That asymmetry is the point.

INTUITION

If a benchmark with three items shows 100% success, you do not believe it is literally infallible. If a benchmark with 3,000 items shows 99.9%, you do. Shrinkage encodes that instinct: small groups borrow strength; large groups stand on their own. The prior strength κ says how many observations a new group needs before you stop borrowing.

IN THIS REPO

`AbilityPrior` in `adaptation.py` is a two-level model: a benchmark-level mean ability plus a subject deviation regressed on the subject embedding, with the ridge penalty chosen by leave-one-out error over a grid $\alpha \in \{1, \ldots, 1000\}$. It is empirical Bayes in spirit: the amount of shrinkage is estimated, not asserted.

TRAP

Estimate μ and κ on training benchmarks only. If you estimate them on the evaluation tasks, you have used the test distribution to set the prior — a subtle form of leakage that is easy to miss because no individual label is touched.

18 Post-hoc calibration in practice

Calibration is fitted on data that looks like deployment but is not deployment: meta-calibration folds from training benchmarks, separately at each budget.

The repo's `AnchorCalibrator` combines two corrections per budget B:

$$p' = (1 - \lambda_B)\,\sigma\big(s_B z + b_B\big) + \lambda_B\,\hat{\pi}_C,$$

where z is the model logit, s_B and b_B are a scale and bias fitted by logistic regression, $\hat{\pi}_C = (1 + \sum_{i \in C} y_i)/(2 + |C|)$ is the smoothed context base rate, and $\lambda_B \in [0, 1]$ is a blending weight chosen on meta-calibration data. At $B = 0$, $\hat{\pi}_C = 0.5$ and the anchor is uninformative; as labels arrive, it becomes a useful level correction.

Why per budget? The miscalibration is not constant: at $B = 0$ the model is often underconfident because it has no subject evidence; at $B = 15$ it may be overconfident because it has fitted the context too closely. One global temperature cannot fix both.

INTUITION

Calibration is a post-processing step, like adjusting a scale that reads two pounds heavy. It does not improve what the scale measures; it makes the number readable. Temperature fixes "too extreme / too flat". The anchor fixes "the whole task is easier or harder than the pool". Both are cheap, both are monotone (so ranking is preserved), and both must be fitted on data you are allowed to see.

SOLVED IN 60 SECONDS

Fitted values: $s = 0.5$, $b = 0.1$, $\lambda = 0.3$, context rate 0.6. Logits $z = -1, 0, 1$.

$z = 1$: $\sigma(0.6) = 0.646$; $p' = 0.7(0.646) + 0.3(0.6) = 0.632$.

$z = 0$: $\sigma(0.1) = 0.525$; $p' = 0.7(0.525) + 0.18 = 0.547$.

$z = -1$: $\sigma(-0.4) = 0.401$; $p' = 0.7(0.401) + 0.18 = 0.461$.

The spread shrank from $(0.269, 0.5, 0.731)$ to $(0.461, 0.547, 0.632)$ and the whole range was pulled toward 0.6. Ordering is unchanged.

TRAP

Fitting the calibrator on the evaluation pool's outcomes is leakage; fitting it on the same fold used to choose the model is optimistic selection. The repo fits on meta-calibration samples from training benchmarks and applies the frozen parameters to held-out tasks. Keep that boundary visible in every experiment.

PART V

Neural Processes

A Neural Process is a function that takes a set of labelled examples and returns a prediction for any new input — in one forward pass, without gradient updates. The family differs only in how much it lets the context summary remember.

- 19 · A distribution over functions
- 20 · Sets and permutation invariance
- 21 · CNP: one deterministic summary
- 22 · NP: the latent variable and ELBO
- 23 · ANP: attention
- 24 · NP-AIRT in this repo
- 25 · Episodic training
- 26 · Reading diagnostics

19 A distribution over functions, not a single function

A Neural Process does not learn one predictor. It learns a rule for turning any small context set into a predictor, and it expresses uncertainty about what happens between the points it has seen.

Ordinary supervised learning fits a function $f : x \mapsto y$ using a large training set. A Neural Process is trained on **tasks**, each of which is itself a small dataset:

$$\text{task} = \big(C,\ T\big), \qquad C = \{(x_i, y_i)\}_{i=1}^{N_C}, \qquad T = \{(x_j, y_j)\}_{j=1}^{N_T}.$$

At meta-training time it sees many tasks. At meta-test time it receives a new context C and must predict y_T for targets x_T. The weights are frozen; only the conditioning changes. This is why the repo calls it **amortized test-time inference**: the adaptation is a forward pass through a set encoder, not a gradient step.

The Gaussian Process is the classical cousin: a GP defines a prior over functions, and conditioning on observations gives a posterior over the function values at new inputs. A Neural Process approximates that conditioning with a neural network, trading exactness for scalability and for the ability to learn from data rather than from a hand-chosen kernel.

INTUITION

You see two points of a curve: $(0, 0)$ and $(1, 1)$. What is the value at $x = 2$? A single-function model must answer. A distribution over functions can say “probably near 2, but it could bend”. Add ten more points and the distribution narrows. A Neural Process learns both the mean answer and how the uncertainty should shrink — that is what makes it a “process”.

SOLVED IN 60 SECONDS

Context: $(0, 0)$ and $(1, 1)$. Query $x = 2$.

A linear model says 2.0 with no uncertainty. A GP with a smooth kernel says about 2.0 with a standard deviation that depends on the kernel length-scale. A CNP says a number determined by its learned decoder; an NP also returns a spread over Monte Carlo samples.

Add the point $(2, 0)$: the linear model is now wrong, the GP posterior bends, and the NP re-encodes the context. The prediction is a function of the *set*, not of a fixed parameter vector.

TRAP

“Meta-learning” here does not mean fine-tuning at test time. If your submission performs gradient steps on the evaluation task, it is doing something else (and probably violating the stateless-call contract). The NP adapts by re-encoding, not by retraining.

20 Sets, pooling, and permutation invariance

The context has no order. A model that cares whether the labels arrived as $(1, 3, 7)$ or $(7, 1, 3)$ is learning noise.

Each context pair is encoded independently, $h_i = e_\phi(x_i, y_i)$. The encodings are then combined by a **permutation-invariant** pooling operator:

$$r = \text{pool}\{h_1, \ldots, h_{N_C}\}, \qquad \text{pool} \in \{\text{mean}, \text{sum}, \max, \text{attention}\}.$$

Mean and sum are the simplest. They are also *decomposable*: the pooled vector is a sum of per-item functions, which is the DeepSets form. This is enough to guarantee that permuting the context permutes nothing in the output. The decoder is then applied to (x_*, r), and it must be permutation-*equivariant* across targets: reorder the query items and the predictions reorder with them.

MASKS ARE NOT AN IMPLEMENTATION DETAIL

Contexts have variable length. If you pad with zeros and take a plain mean, a task with 31 labels is diluted by 31 padded slots. A masked mean divides by the number of real items only:

$$r = \frac{\sum_i m_i h_i}{\sum_i m_i}, \qquad m_i \in \{0, 1\}.$$

The same mask must be applied inside attention (set padded keys to $-\infty$ before the softmax) and inside the loss (ignore padded targets).

INTUITION

A set is a bag, not a list. If you empty a bag onto a table, the order you pull items out does not change what is in the bag. Pooling is the mathematical version of "what is in the bag": a summary that is identical under reordering. Attention is a weighted bag too — the weights move with the items, but the summary does not depend on the order in which they were added.

SOLVED IN 60 SECONDS

Encodings $h_1 = 1,\ h_2 = 2,\ h_3 = 6$. Mean $= 9/3 = 3$.

Permute to $(6, 1, 2)$: mean $= 9/3 = 3$. Same summary.

With a padded fourth slot $h_4 = 0$, the plain mean becomes $9/4 = 2.25$ — a different answer for the same context. The masked mean stays 3.

For attention, the weights reorder with the items but always sum to one; the weighted summary is unchanged.

TRAP

Sorting the context by x before encoding makes the model order-dependent and destroys the set interpretation. So does concatenating a positional index. If a "context encoder" in a paper uses an RNN over the labels, it is not permutation-invariant by construction — it may still work, but it is a different model.

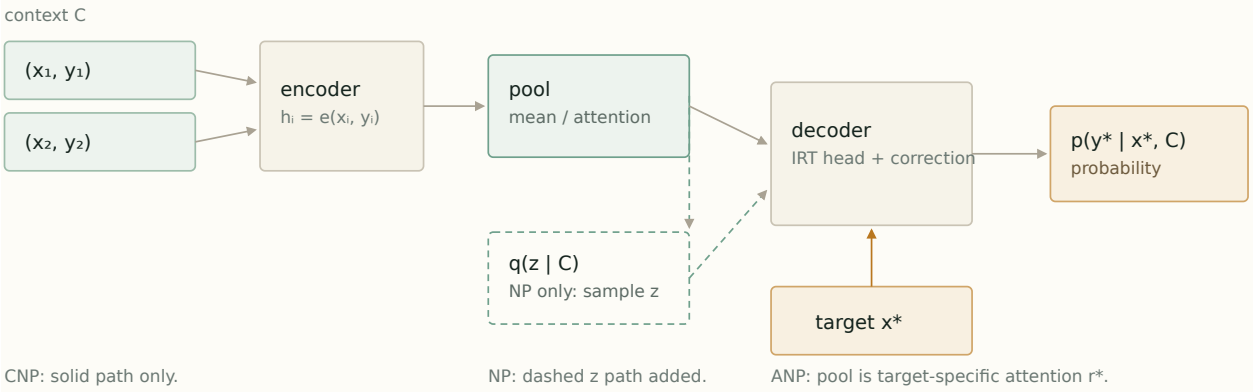
21 CNP: one deterministic summary

A Conditional Neural Process compresses the context into a single vector and decodes every target from that vector. It is fast, simple, and often underfits.

The architecture is three maps:

$$h_i = e_\phi(x_i, y_i), \qquad r = \frac{1}{N_C} \sum_{i=1}^{N_C} h_i, \qquad p(y_* \mid x_*, C) = p_\theta\big(y_* \mid x_*,\, r\big).$$

Training minimizes the negative log-likelihood of target outcomes, averaged over target items and episodes. There is no latent variable and no sampling: the same context always produces the same r, hence the same prediction.



The same skeleton for all three models. The differences are the dashed latent path (NP) and the target-specific attention summary (ANP).

INTUITION

A CNP is a committee that submits one written summary. Every query is answered from that summary. If the context contains two different regimes — easy items and hard items — the mean summary blurs them together, and the CNP cannot say “this target looks like the easy ones”. That blurring is the underfitting that NP and ANP try to fix.

SOLVED IN 60 SECONDS

Encodings $h_1 = (1, 0)$, $h_2 = (0, 1)$; mean $r = (0.5, 0.5)$.

Toy decoder logit $= r_1 - r_2 + 0.5x_*$. For $x_* = 1$: logit $= 0.5 - 0.5 + 0.5 = 0.5$, so $p = \sigma(0.5) = 0.622$.

For $x_* = 0$: logit $= 0$, $p = 0.5$. The only thing that changes is the target input; the context summary is identical for every query.

IN THIS REPO

CNP-AIRT keeps the deterministic context path but replaces the generic decoder with the IRT anchor: $a_j(\theta_s - b_j)$, plus a correction $\Delta\theta(C)$. The ablation ladder asks whether that context encoder earns its keep beyond an item head plus MAP ability — a much sharper question than “does the CNP beat the baseline?”.

22 NP: the latent variable and the ELBO

A latent Neural Process adds a global random variable z that must explain the context. Training pushes the context-only posterior toward the target-aware posterior, and the KL term is what makes the whole thing a variational bound.

A CNP assumes the context summary is sufficient. An NP assumes there is a global task state z — a “mood” that changes how the subject behaves across all items — and that the context is noisy evidence about it:

$$z \sim p(z) = \mathcal{N}(0, I), \qquad y_* \mid x_*, z \sim p_\theta(y_* \mid x_*, z).$$

The posterior $p(z \mid C)$ is intractable, so we approximate it with a network $q_\phi(z \mid C) = \mathcal{N}(\mu_\phi(C), \operatorname{diag}\sigma^2_\phi(C))$. The log marginal likelihood is bounded below by the ELBO:

$$\log p(y_T \mid x_T, C) \;\geq\; \mathbb{E}_{q_\phi(z|C)}\big[\log p_\theta(y_T \mid x_T, z)\big] - \mathrm{KL}\big(q_\phi(z \mid C) \,\|\, p(z)\big).$$

At training time the targets y_T are available, so we can use a sharper posterior $q_\phi(z \mid T)$. The standard NP objective is

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|T)}\big[-\log p_\theta(y_T \mid x_T, z)\big] + \frac{1}{N_T}\,\mathrm{KL}\big(q_\phi(z \mid T) \,\|\, q_\phi(z \mid C)\big).$$

The KL term forces the context-only posterior to match the target-aware one. At inference only C exists, so $q_\phi(z \mid C)$ is used. Averaging over L samples of z gives the predictive probability; the spread across samples is the model’s epistemic uncertainty.

INTUITION

Think of z as the "subject's mood today": a single hidden state that shifts the difficulty of every item in the same direction. The context is a few observed answers; the posterior is your belief about the mood. The KL term is a consistency requirement: whatever mood explains the context should also explain the targets. Without it, the model could use a different mood for every target and learn nothing general.

SOLVED IN 60 SECONDS

KL between $\mathcal{N}(1, 0.25)$ and $\mathcal{N}(0, 1)$:

$\log(1/0.5) = 0.693$; $(0.25 + 1)/2 = 0.625$; minus 0.5 gives 0.818 nats.

Two target items with predicted probabilities 0.8 and 0.3, outcomes 1 and 0: NLL $= -[\log 0.8 + \log 0.7]/2 = (0.223 + 0.357)/2 = 0.290$.

Total loss $= 0.290 + 0.818/2 = 0.699$. The KL is divided by N_T, so it does not dominate when there are many targets.

IN THIS REPO

The v1 objective used a prior-KL with weight $1/|T|$ and an "escape hatch"; the latent collapsed. The v2 objective is the standard (A)NP objective above with weight 1 on the dual likelihood, and disjoint C, T. Even so, the measured $\mathrm{KL}(q_T \| q_C) \approx 2 \times 10^{-7}$ and effective latent rank 1 when z enters only through a scalar θ. That is the repo's central negative result: a global latent is redundant when the analytic MAP already owns the only global degree of freedom.

TRAP

A KL that goes to zero is not necessarily good news. It can mean the posterior collapsed to the prior and z is being ignored. Check the KL, the effective rank of the Jacobian, and whether predictions change when z is resampled. If none of them move, you have an expensive CNP.

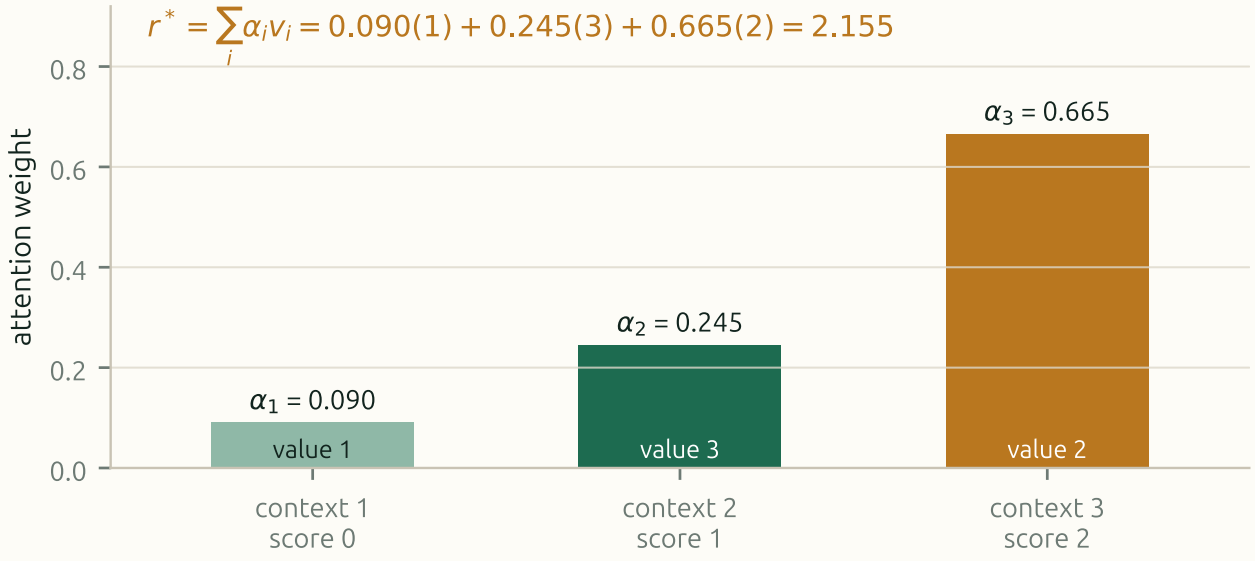
23 ANP: attention makes the summary target-specific

Instead of one summary for all queries, an Attentive Neural Process lets each target look at the context items that resemble it and build its own summary.

Each context item provides a key k_i and a value v_i; each target provides a query q_*. The attention weights and the target-specific representation are

$$\alpha_{*i} = \frac{\exp\left(q_*^\top k_i / \sqrt{d}\right)}{\sum_{i'} \exp\left(q_*^\top k_{i'} / \sqrt{d}\right)}, \qquad r_* = \sum_{i \in C} \alpha_{*i} \, v_i.$$

The $\sqrt{d}$ keeps the logits from exploding as the dimension grows. With multiple heads, several such attention maps run in parallel and are concatenated; with self-attention, context items also attend to each other before the cross-attention, which lets the model represent interactions among items.



Softmax attention is a convex combination. The weights are positive and sum to one, so r^* is a weighted average of context values. A peaked distribution selects one item; a flat one averages the whole context.

WHAT ATTENTION BUYS, AND WHAT IT COSTS

- **Buys:** target-specific summaries. A hard target can attend to hard context items instead of the global mean.
- **Buys:** permutation equivariance for free — the sum over context items does not depend on their order.
- **Costs:** $O(N_C N_T)$ scores, plus a second path to train. On 31 context items this is trivial; on 10,000 it is not.
- **Costs:** attention weights are not explanations. A high weight means "useful for this prediction", not "caused this prediction".

INTUITION

A CNP asks the whole class for one answer. An ANP lets each student ask the classmates who have done similar problems. The query is the target, the keys are the context items, and the values are what those items know. Softmax decides how much of each classmate's answer to use. The result is still a set operation, but a much more flexible one.

SOLVED IN 60 SECONDS

Scores $q^\top k_i/\sqrt{d} = (0, 1, 2)$. Softmax $= (0.090, 0.245, 0.665)$. Values $(1, 3, 2)$.

$r^* = 0.090(1) + 0.245(3) + 0.665(2) = 0.090 + 0.735 + 1.330 = 2.155.$

If the target were identical to context item 3, the score gap would grow and the weight would approach 1; r^* would approach value 2. Attention can interpolate between "listen to everyone" and "listen to one".

TRAP

Padding is the classic silent bug. If padded context slots are not masked to $-\infty$ before the softmax, every target attends to zeros and the weights no longer sum over real items. The model still trains, which is what makes it dangerous.

24 NP-AIRT in this repo: where the anchor lives

The repo's models are Neural Processes with an IRT decoder: the network learns item parameters and a correction, while a MAP ability sets the level.

The decoder combines the psychometric anchor with a learned residual:

$$\theta_s = \mathrm{MAP}(C), \qquad \ell_{sj} = a_j(\theta_s - b_j) + \delta_\omega(x_j, z, r_j^*), \qquad p_{sj} = \sigma(\ell_{sj}).$$

Three design choices matter more than the architecture:

1. θ_s **is not a network output.** It is the analytic MAP of §13. The network owns the item side and the correction; the anchor owns the level. This prevents the latent from collapsing into a scalar and keeps ability comparable across tasks.
2. **The residual is bounded and mean-centered across targets.** $\delta = \delta_{\max}\tanh(\cdot)$, centered so it can reorder items without changing the overall level. Without centering, "correction" and "ability" become the same knob.
3. **A budget-0 gate.** At $B = 0$, $\Delta\theta = 0$ and $\delta = 0$, so every adaptive variant returns the identical AIRT prediction. Otherwise the empty-context prior would silently define a different zero-budget model.

SOLVED IN 60 SECONDS

Item $a = 1.2$, $b = 0.2$; subject $\theta = 0.5$; correction $\delta = 0.4$.

Anchor logit $= 1.2(0.5 - 0.2) = 0.36$, so $p = \sigma(0.36) = 0.589$.

With the correction: logit $= 0.36 + 0.4 = 0.76$, $p = \sigma(0.76) = 0.681$.

At $B = 0$ the gate sets $\delta = 0$, so the answer is 0.589. That is the number the model must reproduce with an empty context, by construction.

IN THIS REPO

`src/aims_starter/faithful_np.py` implements NP, ANP, and stacked-MHSA-ANP variants with an IRT-anchored decoder. `modal_cnp_ladder.py` runs the encoder-contribution ladder (frozen 1PL, frozen 2PL, MAP, CNP, minus-correction control). The naming is deliberate: CNP-AIRT is deterministic; NP-AIRT is latent; ANP-AIRT is attentive.

TRAP

If the residual is unbounded or uncentered, it can absorb the subject's ability and the item's difficulty at the same time. The model still fits, but "ability" is no longer identified, and the IRT story in the paper becomes false. Bound it, center it, and ablate the gate.

25 Training an episodic model without fooling yourself

The training loop is where the competition's constraints become code: disjoint context and targets, variable context sizes, and a loss that matches the deployment query.

One episode samples a task, a context size from the official budget grid, a context subset, and a disjoint target subset. The loss is the NP objective of §22, averaged over target items. Three details decide whether the model generalizes:

- **Disjoint C and T.** If a target appears in the context, the model can copy the label and the training NLL becomes meaningless. The corrected repo episodes use disjoint sets; the older image-style $C \subseteq T$ convention was a bug for this task.
- **Context-size distribution.** Train on sizes drawn from $\{0, 1, 3, 7, 15, 31\}$ so the model sees the same sparsity it will face. Training only at 31 and testing at 1 is a different task.
- **Masking.** Variable-length contexts and targets need masks in the encoder, the attention, and the loss. A padded target must not contribute a gradient.

Optimization is ordinary: Adam, gradient clipping, deterministic seeds, early stopping on held-out *benchmarks*. What is not ordinary is the evaluation unit — a held-out benchmark, not a held-out row.

INTUITION

Episodic training is a fire drill. Each episode is a miniature of the real job: here are a few labelled items, now predict these unseen ones. If the drill always gives you 31 labels and the real job gives you one, you have trained for the wrong job. If the drill lets you peek at the answer sheet, you have trained for a job that does not exist.

SOLVED IN 60 SECONDS

Two targets, predicted $p = (0.8, 0.3)$, outcomes $y = (1, 0)$.

NLL $= -[\log 0.8 + \log 0.7]/2 = 0.290$.

KL $= 0.818$, $N_T = 2$, so the KL term is 0.409. Total $= 0.699$.

If the KL were not divided by N_T, it would contribute 0.818 and dominate the likelihood. The $1/N_T$ convention keeps the two terms on a comparable scale as the target set grows.

TRAP

Do not early-stop on the test benchmark, and do not choose the checkpoint by Codabench score and then report that score as evidence. The public leaderboard is a test set; using it for model selection turns it into a validation set and inflates the reported number.

26 Reading the diagnostics: signal, scale, or noise?

Most "the model is broken" moments are one of three things: no signal, wrong scale, or too little data to tell. The metrics tell them apart if you read them together.

PATTERN	LIKELY CAUSE	FIRST FIX
Brier ≈ 0.25, AUC ≈ 0.5	no item signal; predictions near the base rate	check features, leakage-free split, and the item encoder
AUC 0.63, Brier 0.27	ranking works, probability scale is wrong	temperature / anchor calibration; inspect mean p vs base rate
ECE high, AUC fine	miscalibration only	per-budget calibration; do not touch the architecture
KL ≈ 0, rank 1	latent collapsed or redundant	remove z; test a deterministic CNP
Train NLL falls, val Brier rises	overfitting or distribution shift	benchmark-held-out validation; stronger regularization
Bump at $B = 1$, recovers by $B = 7$	overreaction to one label	shrinkage / prior strength; check the update rule
Seed spread $\gg$ model gap	underpowered comparison	more seeds, paired tasks, report intervals

Two quantities are especially useful and often skipped. The **mean predicted probability** should be close to the observed base rate on a held-out task; a gap is a level error, which calibration can fix. The **effective rank** of the latent Jacobian tells you whether z is doing anything at all: rank 1 means the latent is a scalar in disguise.

SOLVED IN 60 SECONDS

A model has AUC 0.63, Brier 0.27, mean prediction 0.72, base rate 0.50.

It ranks better than chance but predicts 22 points too high. On 100 balanced items, the Brier penalty from level alone is roughly $(0.72 - 0.5)^2 = 0.048$.

Apply a bias correction that maps the mean to 0.50 (or fit a temperature on meta-calibration data): Brier can drop toward 0.22 without changing AUC at all. The signal was there; the scale was not.

Read this: never conclude “the model has no signal” from Brier alone. Check AUC and the mean prediction first.

INTUITION

Separate the two questions. *Can the model tell items apart?* That is AUC / resolution. *Does it say the right number?* That is calibration / reliability. Brier is the product of both, so a bad Brier score is ambiguous until you look at the pair. Debugging in the wrong order wastes days.

PART VI

The competition playbook

A ladder tells you what to build next. An ablation tells you whether it worked. A checklist tells you why the leaderboard disagrees. And a cram sheet gets you through the meeting.

- 27 · The model ladder
- 28 · Ablations that survive review
- 29 · Failure-mode checklist
- 30 · Cram sheet

27 The model ladder: what each rung actually tests

Every added component must answer a question the previous rung could not. If it cannot beat the simpler control, it is not a contribution.

RUNG	WHAT IT ADDS	QUESTION IT ANSWERS
Constant 0.5	nothing	Is the model better than admitting ignorance? (Brier 0.25)
Beta-Binomial	one online rate	Can a conjugate update with no item content beat the constant?
1PL / 2PL	per-item difficulty (± discrimination)	Is there item structure worth modeling?
Amortized IRT	$f_\phi(x_j) \mapsto (a_j, b_j)$	Can content predict difficulty for unseen items?
Online-AIRT	MAP θ from acquired labels	Is a scalar Bayesian update enough?
CNP-AIRT	deterministic context encoder + correction	Does learned global conditioning beat the analytic update?
NP-AIRT	global latent z	Does a stochastic task state add anything?
ANP-AIRT	target-specific attention r^*	Does item-to-item relevance help beyond a global summary?
Stacked ANP	self-attention over context	Do context–context interactions help?
RAA-IRT	explicit relational features	Is a gain genuinely relational, or just more parameters?
Shuffled relations	negative control	Does destroying the relation structure remove the gain?

The ladder is cumulative for a reason. If ANP-AIRT beats CNP-AIRT but NP-AIRT does not, the gain is attention, not the latent. If RAA-IRT beats ANP-AIRT but the shuffled-relation control does too, the gain is capacity, not relations.

INTUITION

A ladder is a sequence of controlled comparisons, not a wish list. Each rung isolates one degree of freedom: item content, then subject adaptation, then a global latent, then target-specific attention. If you add two rungs at once and the score improves, you have learned that *something* helped — which is not a scientific claim.

SOLVED IN 60 SECONDS

Model A ALC 0.224, Model B ALC 0.219, difference 0.005. Seed spread across three seeds: ± 0.006.

Paired per-task differences: mean -0.005, SE $= 0.004$. A 95% interval is roughly $[-0.013, 0.003]$ — it crosses zero.

Decision: do not claim B is better. Either run more seeds/tasks or report the comparison as inconclusive. A 0.005 gain with 0.006 noise is not a result.

IN THIS REPO

The named ladder is Online-AIRT → CNP-AIRT → NP-AIRT → ANP-AIRT → stacked ANP → per-item latent correction → hybrid anchor. The negative controls are a correction-free CNP item head, a latent with $d_z = 1$, shuffled acquisition orders, and shuffled relations. They are part of the argument, not an appendix.

28 Ablations that survive review

An ablation is an experiment, not a victory lap. Freeze everything you are not testing, change one thing, and decide the rule before you see the numbers.

The repo's binding protocol is a good template:

- **Fixed task grid.** 104 validation tasks, with eligible subject–benchmark pairs defined in advance.
- **Frozen acquisition orders.** Ten fixed candidate streams, one seed for the build, so seeds cannot disagree on the labels.
- **Nested budgets.** The budget-3 context contains the budget-1 label, and so on.
- **Out-of-fold calibration.** Five folds; calibrator parameters for a task never see that task's outcomes.
- **One test read.** The test fold is opened once, for the selected model and the baseline, after selection is complete.

Report paired per-task differences, not just two means. Report every seed, not the best one. Report benchmark-level failures, runtime, and parameter count — a gain that costs 100× compute is a different result from a free one.

SOLVED IN 60 SECONDS

Question: does attention help beyond the global summary?

Design: same AIRT base, same episodes, same budgets, same calibration folds, same seeds. Arm A replaces r^* with the mean-pooled r; Arm B uses attention. Everything else is identical.

Metric: paired ALC difference per task, 10 frozen orders, report mean, 95% interval, and the fraction of tasks improved.

Decision rule (written first): claim a gain only if the interval excludes zero and the gain survives the shuffled-relation control. Otherwise report "no detectable difference".

TRAP

If each arm gets its own hyperparameter search, you are comparing tuning budgets, not architectures. Fix the budget, or tune on a nested split and report the search cost.

IN THIS REPO

`docs/ablation-protocol.md` is the contract; `docs/uniform-audit.md` checks that the code actually follows it. The audit exists because an ablation protocol that is only described, not enforced, drifts.

29 Failure-mode checklist

When the offline number and the leaderboard disagree, the cause is almost always on this list.

1 · Target leakage. Do evaluation outcomes enter any fit, calibration, early stop, or order choice? Check the call graph, not just the model.

2 · Subject identity in the item path. Are subject embeddings masked from a_j, b_j? If not, "difficulty" is subject-specific.

3 · Duplicate items. Do paraphrases or repeated trials cross the train/test split? Group by item and benchmark.

4 · Non-nested budgets. Does budget 7 contain the labels from 1 and 3? If not, the curve is not a learning curve.

5 · Micro vs macro. Are you averaging items when the leaderboard averages tasks? See §11.

6 · Calibration on test. Was the calibrator fitted on the evaluation tasks? It must be fitted elsewhere.

7 · AUC-only tuning. Did you select a checkpoint by AUC and then report Brier? Ranking and scale are different.

8 · Seed cherry-picking. Is the reported seed the best of several? Report all seeds and the spread.

9 · Budget-0 mismatch. Does the model at $B = 0$ reproduce the frozen anchor? The gate should force it.

10 · Padding bugs. Are padded context slots masked in the mean, the softmax, and the loss? Check a task with $B = 1$.

11 · Dead softmax. Are attention logits saturated or NaN? Inspect weights and gradients.

12 · KL collapse. Is z ignored? Resample z and see whether predictions move.

13 · Statelessness. Does the predictor rebuild from the full labelled list each call? Workers may restart between budgets.

14 · Probability clipping. Are 0 and 1 replaced by epsilon before the log? Otherwise one confident mistake produces infinity.

15 · Benchmark shift. Is the test benchmark unlike training? Report per-benchmark failures, not one average.

SOLVED IN 60 SECONDS

Offline 0.18 vs leaderboard 0.31: too large for seed noise. Check calibration first (mean 0.7 vs base 0.4 means wrong scale), then the split (duplicates, subject identity, calibration on evaluation).

30 Cram sheet

Everything on one page, for the ten minutes before the meeting.

FORMULAS

logit: $z = \log \dfrac{p}{1-p}$, $\qquad p = \sigma(z) = \dfrac{1}{1+e^{-z}}$, $\qquad \sigma'(z) = \sigma(z)(1-\sigma(z))$

Bernoulli NLL: $-\dfrac{1}{n}\sum_i \big[y_i \log p_i + (1-y_i)\log(1-p_i)\big]$

Beta--Bernoulli: $\pi \sim \mathrm{Beta}(\alpha,\beta)$, $\ \pi \mid y \sim \mathrm{Beta}(\alpha+k,\ \beta+n-k)$

2PL: $p_{sj} = \sigma\big(a_j(\theta_s - b_j)\big)$, $\qquad$ 3PL: $c_j + (1-c_j)\sigma\big(a_j(\theta_s - b_j)\big)$

MAP ability: $g = \sum_i a_i(y_i - p_i) - \dfrac{\theta-\mu_0}{\sigma_0^2}$, $\quad h = -\sum_i a_i^2 p_i(1-p_i) - \dfrac{1}{\sigma_0^2}$

Brier: $\mathrm{BS} = \dfrac{1}{n}\sum_i (p_i - y_i)^2$, $\qquad \mathbb{E}[(p-Y)^2] = (p-\pi)^2 + \pi(1-\pi)$

ALC: $0.1B_0 + 0.2B_1 + 0.2B_3 + 0.2B_7 + 0.2B_{15} + 0.1B_{31}$

ECE: $\sum_k \dfrac{n_k}{n}|\bar{p}_k - \bar{y}_k|$, $\qquad \mathrm{AUC} = P(s_+ > s_-) + \frac{1}{2}P(s_+ = s_-)$

ELBO: $\mathbb{E}_{q(z\mid C)} \log p(y_T \mid z) - \mathrm{KL}\big(q(z \mid C) \,\|\, p(z)\big)$

Gaussian KL: $\log \dfrac{\sigma_2}{\sigma_1} + \dfrac{\sigma_1^2 + (\mu_1-\mu_2)^2}{2\sigma_2^2} - \dfrac{1}{2}$

Attention: $\alpha_i = \dfrac{\exp(q^\top k_i/\sqrt{d})}{\sum_{i'}\exp(q^\top k_{i'}/\sqrt{d})}$, $\qquad r^* = \sum_i \alpha_i v_i$

NUMBERS TO REMEMBER

Brier of a constant 0.5	**0.25** — the “knows nothing” bar
Budgets and weights	0, 1, 3, 7, 15, 31; 0.1, 0.2, 0.2, 0.2, 0.2, 0.1
Repo ability prior	$\mathcal{N}(0,4)$, clamped ± 4, 12 Newton steps
Repo Beta-Binomial	$\mathrm{Beta}(2,2)$, $\hat{p} = (2+k)/(4+n)$
Entropy at $p = 0.5$	0.693 nats (1 bit)
Logit landmarks	0 → 0.50; 1 → 0.73; 2 → 0.88; 4 → 0.98
Odds multiplier	+1 logit multiplies odds by $e \approx 2.72$
90% interval width	≈ 3.29 standard deviations

DIAGNOSIS IN ONE LINE EACH

- **Brier 0.25, AUC 0.5:** no signal. Check features and leakage.
- **AUC high, Brier bad:** wrong scale. Calibrate; do not retrain.
- **ECE high, AUC fine:** miscalibrated only. Temperature or anchor.
- **KL ≈ 0, rank 1:** latent redundant. Use a CNP or reparameterize.
- **Validation great, test bad:** leakage or benchmark shift. Check the split first.
- **Bump at B = 1:** overreaction. Increase shrinkage.

TEN QUESTIONS

1. Why is 0.25 the Brier baseline? A constant 0.5 gives $(0.5)^2$ on every item.
2. Why can AUC rise while Brier falls? AUC is rank-only; Brier also cares about scale.
3. What does a prior do at $B = 1$? Prevents infinite ability and shrinks the update.
4. Why divide KL by N_T? Keeps it comparable to the per-target likelihood as targets grow.
5. What makes a CNP permutation-invariant? Per-item encoding plus a symmetric pool.
6. What does ANP add? A target-specific context summary via attention.
7. Why is 3PL skipped? Guessing is not identifiable from 31 labels and has no mechanism here.
8. Why is a paired test better? Task difficulty cancels; the comparison is more precise.
9. Why macro-average? The leaderboard weights subject–benchmark tasks equally.
10. Why freeze the acquisition order? To separate predictor quality from label-stream luck.

> **THE ONE SENTENCE**
>
> Predict a probability for every hidden outcome, use content for the cold start, use a few labels to update a low-dimensional state, and never let the evaluation outcomes touch the fit.

APPENDIX

Glossary and map

The vocabulary, the files, and the papers — in the order you are likely to need them.

- A · Glossary
- B · Where things live
- C · Reading list

A Glossary

The vocabulary of the competition, in the order it usually bites.

Acquisition pool	items whose outcomes may be revealed; disjoint from the scored evaluation pool.
ALC	area under the Brier learning curve; here a fixed weighted average over six budgets.
Amortized inference	learning a network that predicts the result of an inference step in one forward pass.
ANP	Attentive Neural Process; uses cross-attention to build a target-specific summary.
AUC	probability that a random positive is ranked above a random negative; rank-only.
Beta-Binomial	conjugate model for a binary rate with a Beta prior; the repo's control predictor.
Brier score	mean squared error of probabilities; strictly proper; lower is better.
Calibration	agreement between predicted probabilities and observed frequencies.
CNP	Conditional Neural Process; deterministic context summary, no global latent.
Context C	the labelled items revealed so far; the conditioning set.
Discrimination a_j	steepness of an item's characteristic curve; how sharply it separates abilities.
Difficulty b_j	the ability level at which success probability is 0.5 (2PL).
ECE	expected calibration error over probability bins; diagnostic, not proper.
ELBO	evidence lower bound; the variational objective used to train latent NPs.
Episode	one meta-training task: a context set plus target set drawn from a task.
Evaluation pool	items on which predictions are scored; outcomes hidden.
Identifiability	whether the data can distinguish parameter values that predict identically.
IRT	item response theory; models binary outcomes from ability and item parameters.
KL divergence	non-negative, asymmetric cost of using q when the truth is p, in nats.
Logit	$\log(p/(1-p))$; the linear scale of the model.
MAP	maximum a posteriori; the parameter value maximizing posterior density.
Meta-learning	learning across tasks so that a new task can be handled from a small context.
Nats	unit of log-probability; 1 nat = 1.4427 bits.
NP	Neural Process; a CNP with a global latent variable z.
Permutation invariance	output unchanged when the input set is reordered.
Proper scoring rule	a score whose expectation is optimized by reporting the true probability.
Resolution	the part of Brier that rewards separating easy from hard cases.
Shrinkage	pulling a noisy estimate toward a pooled value; partial pooling.
Subject	the AI system being evaluated, not the predictor.
Temperature T	divisor on the logit; $T > 1$ softens, $T < 1$ sharpens.
2PL	two-parameter logistic IRT: ability, difficulty, discrimination.

B Where things live in the repo

Twelve files, and the question each one answers.

FILE	WHAT IT DOES
`src/aims_starter/metrics.py`	Brier, ECE, AUC, and the official ALC weights. Read this first.
`src/aims_starter/adaptation.py`	Newton MAP for θ, benchmark ability prior, ridge subject deviation.
`src/aims_starter/calibration.py`	per-budget scale/bias/anchor calibration fitted on meta-calibration folds.
`src/aims_starter/faithful_np.py`	NP, ANP, and stacked-MHSA-ANP with the IRT-anchored decoder.
`src/aims_starter/data.py`	episode construction, feature slices, and split handling.
`src/aims_starter/evaluation.py`	budgeted evaluation loop and aggregation.
`submission/model.py`	the self-contained predictor packaged for Codabench.
`docs/notation.md`	canonical symbols; use it before naming anything.
`docs/methods-guide.md`	the one-page method summary for the paper.
`docs/raa_irt_design.md`	the proposed relation-aware extension and its controls.
`docs/ablation-protocol.md`	the binding experiment contract; read before running anything.
`docs/uniform-audit.md`	the audit that checks the code against the contract.

HOW TO USE THE MAP

If you have five minutes before a meeting, open `metrics.py` and `methods-guide.md`. If you have an hour, read `faithful_np.py` alongside §21–25. If you are debugging a number, start at `evaluation.py`, not at the model — aggregation bugs look like model bugs.

C Reading list

Four papers cover almost everything in this guide. Read them in this order; each one assumes the previous.

PAPER	READ FOR	ONE-LINE SUMMARY
Conditional Neural Processes Garnelo et al., 2018 · arXiv:1807.01613	the CNP skeleton	Encode each context pair, pool, decode each target. No latent.
Neural Processes Garnelo et al., 2018 · arXiv:1807.01622	the latent + ELBO	Add a global z and train with a context/target KL.
Attentive Neural Processes Kim et al., 2019 · arXiv:1901.05761	attention variants	Target-specific summaries; dot-product, multihead, stacked self-attention.
Amortized IRT Truong et al., 2025 · PMLR v267	the competition's base model	Predict item parameters from item content; fit ability from a few responses.

For the statistics, any standard treatment of proper scoring rules and calibration will do; the Brier decomposition in §7 is the Murphy decomposition. For shrinkage, the empirical-Bayes chapter of Efron & Hastie is the canonical reference. For the competition protocol, the official starter kit and `docs/ablation-protocol.md` are more authoritative than any summary here.

A LAST WORD

You do not need to hold all of this in your head. Hold the one sentence: *a subject's ability, compared with an item's difficulty, squashed into a probability — with the ability updated from a few labels and the difficulty predicted from content.* Everything else in this guide is a detail hanging off that sentence.